



MASTERING COMMUNICATIONS ENABLEMENT

with Avaya ACE™ Web Services



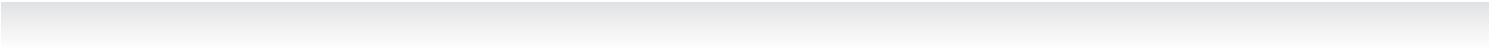
CORE IP COMMUNICATIONS ENABLEMENT FOR IT DEVELOPERS

TABLE OF CONTENTS

INTRODUCTION	1
About this eBook	1
About communications enabled business systems and Avaya ACE.....	1
Why you should read this eBook	2
Avaya ACE releases	3
Assumed knowledge	3
About the Avaya DevConnect program.....	3
How the eBook is organized.....	3
Icons used in this eBook.....	5
 CHAPTER 1: ABOUT AVAYA ACE AND AVAYA ACE WEB SERVICES	 6
What is Avaya ACE?	6
Application enablement interfaces and toolkits	6
Avaya ACE middleware	7
What differentiates Avaya ACE Web services from other CEBS interfaces?	9
Avaya ACE user profiles.....	10
Avaya ACE Web services.....	12
Communications network infrastructure and vendor neutrality.....	13
Other key features of Avaya ACE	14
Unified Communications capabilities.....	14
Federated and high availability configurations	16
Security features.....	16
Round-up of Chapter 1.....	16
 CHAPTER 2: AN INTRODUCTION TO WEB SERVICES AND PARLAY X	 17
What are Web services?	17
Types of Web service.....	18
The Avaya ACE Web Services.....	21
Web service definition files.....	21
Web services client development	25
Parlay X	26
Parlay X compliant Web services	27
Round-up of Chapter 2	28

CHAPTER 3: AVAYA ACE REAL-TIME COMMUNICATIONS SERVICES.....	29
Making voice calls.....	29
Call participant addresses.....	30
Avaya ACE Web services used to make calls.....	31
Controlling ongoing voice calls.....	32
Avaya ACE Web services used to control calls.....	33
Creating and controlling conferences.....	34
Ad hoc conferences.....	34
Avaya ACE Web services used to create and control conferences.....	35
Playing prerecorded announcements.....	35
Avaya ACE Web services used to play announcements.....	36
Managing and using presence information.....	36
Avaya ACE Web services used to get and manage presence information.....	38
Setting up and managing call forwarding rules.....	38
Avaya ACE Web services used to set up and manage call forwarding rules.....	38
Round-up of Chapter 3.....	38
 CHAPTER 4: AVAYA ACE INFORMATION AND SYSTEM SERVICES.....	 39
Information services.....	39
Getting call history information.....	39
Getting information about the location of communications devices.....	40
System services.....	40
Creating and managing Avaya ACE users, user groups and global security settings.....	40
Getting Avaya ACE subscriber information.....	41
Monitoring the health of Avaya ACE systems.....	42
Round-up of Chapter 4.....	42
 CHAPTER 5: DEVELOPING AVAYA ACE WEB SERVICES CLIENTS.....	 43
Avaya ACE development configurations.....	43
Obtaining Avaya ACE WSDL.....	43
Generating client code from WSDL.....	44
Using Apache Axis to generate a Java client proxy.....	44
Exploring the generated client proxy code.....	47
Writing client application code.....	53
Running the example client application code.....	61
Round-up of Chapter 5.....	62

CHAPTER 6: WEB CLIENT DEVELOPMENT KNOW-HOW	63
Redirect unanswered calls solution	64
Start call notification.....	64
Receive call notifications	66
Stop call notification	68
First aid personnel presence solution	68
Subscribe for presence.....	69
Start presence notification.....	70
Receive presence status information	71
Display presence information	72
Enable click-to-dial links	73
Round-up of Chapter 6.....	73
 CHAPTER 7: AVAYA ACE PACKAGED APPLICATIONS.....	74
Avaya ACE applications and UC add-ins	74
ACE Integration Engine applications	74
Other Avaya ACE applications.....	75
UC Add-ins.....	75
Event Response Manager application	76
Event Response Manager Administration.....	76
Generating Event Triggers	77
Round-up of Chapter 7	77
 CHAPTER 8: NEXT STEPS.....	78
Join the DevConnect program.....	78
Learn more.....	78
Find out which capabilities are supported in your environment	78
Choose your application development tools.....	79
Develop and test your client code	79
Get no cost technical support	79
Compliance test your solution (higher level DevConnect members only)	79
Round-up of Chapter 8.....	80
 APPENDIX A: AVAYA ACE WEB SERVICES AND OPERATIONS	81



APPENDIX B: AVAYA DEVCONNECT DEVELOPER AND PARTNER PROGRAM86

Ready for more?86

About the DevConnect program86

A sample listing of available DevConnect resources.....87

 SDK and API downloads and documentation..... 87

 Remote labs..... 87

 Training courses, tutorials and technical webinars.....88

 Tools, evaluation kits and developer editions88

 Sample applications88

 DevConnect eBooks.....88

Register today.....88

APPENDIX C: COMPARISON OF PROGRAMMING INTERFACES89

REFERENCES..... 91

INTRODUCTION

About this eBook

Mastering Communications Enablement with Avaya ACE™ Web Services is an updated and expanded version of the DevConnect eBook, *Exploring the Communications Enablement of Business Systems with Avaya Agile Communication Environment™ (ACE) Web Services*. This edition includes new chapters on Web services and development tools, on developing Avaya ACE Web service clients, on using Avaya ACE Web services in real-life solutions, and on the packaged Avaya ACE applications and add-ins. The eBook has also been updated to reflect changes in the latest release of Avaya ACE available at the time of publication (see *Avaya ACE releases* below).

About communications enabled business systems and Avaya ACE

The focus of this eBook is on the use of Avaya Agile Communication Environment™ (Avaya ACE™) core Web services to access communications capabilities and blend them into Communications Enabled Business Systems (CEBS).

A common issue faced by many enterprises is human latency in business processes. A high level of collaboration and interaction is needed to make and communicate decisions at the various steps in a business process. Latency introduced at each step increases costs, reduces time to market, and can greatly reduce the overall customer experience.

In a Communications Enabled Business System, automated communications are tightly integrated into business process flows. Whilst a Communications Enabled Business System might leverage Unified Communications (UC) capabilities, such as Click-to-contact and presence, CEBS applications are usually triggered and driven by business events. For example, when the price of a specified commodity breaches a predefined threshold a trading application automatically contacts the personnel who are available (as identified by their presence status) and can take action (as identified by their skill set) to execute the next step in the process. Communications Enabled Business Systems incorporate communications capabilities directly into the sequence of tasks in business processes. By enabling communications to be triggered directly from the business system, communications can be sped up significantly, thus reducing human latency and costs, and improving efficiency and effectiveness.

Communications Enabled Business Systems allow enterprises to streamline workflows, quickly respond to time-sensitive issues and improve customer satisfaction by ensuring the right people, armed with the right information, are able to respond at the right time.

Avaya ACE supports both UC and event-driven CEBS capabilities. It is designed to help IT developers inject communications capabilities into their enterprise business systems, without having to acquire detailed and highly specialized IP communications development knowledge. To facilitate the development of CEBS, Avaya ACE:

- Provides a simple programming interface that allows developers to access core capabilities that satisfy most of the communications requirements of business systems. Avaya ACE enables client applications to, for example:
 - ◇ Make and control calls.
 - ◇ Create conferences and manage participants.
 - ◇ Play audio messages to users on their communications devices.
 - ◇ Access and manage user presence information.
- Provides access to communications capabilities via a choice of high level Web services APIs or low level Foundation Runtime Services. We'll look at the differences between these programming interfaces in *Chapter 1*, but suffice to say for now that this eBook is primarily about using the Avaya ACE Web services.

Web services are a widely used technology, well known to most business system developers and programming language independent. The Avaya ACE Web services are based on open standards.

- Provides Web services that work with a wide variety of underlying communications network infrastructures provided by a variety of vendors.

Why you should read this eBook

This eBook is a great place to start to learn about the benefits of communications enabling business systems. It provides an ideal introduction to the use of Avaya ACE Web services for anyone involved in the development of business systems, including IT software engineers, systems and business analysts, systems architects and IT managers working in enterprise environments. You will learn about the communications capabilities enabled by Avaya ACE Web services and how these can be employed in practical business solutions. You will also discover how quickly and easily you can incorporate communications into your business systems using the Avaya ACE Web services.

Avaya ACE releases

The information in this eBook is based on Avaya ACE Release 2.3.1, which is the current release at the time of publication (December 2010).

Assumed knowledge

Like the Avaya ACE Web services themselves, this eBook is targeted at the IT development community and does not assume or require detailed IP communications programming knowledge. To get the most out of this book it would be useful to have some experience of developing Web service clients using SOAP and HTTP, although this is not essential.

About the Avaya DevConnect program

DevConnect is Avaya's developer and partner program, open to IP communications application developers, System Integrators, ISVs, IHVs and customers alike. Basic Registered level membership is free, offering no-charge access to educational materials, product API documentation, Software Development Kits (SDKs), sample applications, technical support and more.

A variety of tools and other resources are available to DevConnect members for Avaya ACE Web services client application development, including programming documentation, a remote lab for self-testing applications under development and access to technical support.

Companies that wish to extend their relationship with Avaya can apply to become an enhanced-level member, which offers greater technical support, discounted procurement for lab systems and co-marketing benefits based on proven compliance with Avaya Solutions.

For more information, see *Appendix B* at the end of this eBook or visit the DevConnect portal now and sign-up for free registered membership.

How the eBook is organized

This eBook comprises the following chapters and appendixes:

Chapter 1: About Avaya ACE and Avaya ACE Web services

Chapter 1 discusses what Avaya ACE is and how it bridges the gap between an organization's business systems and its IP communications infrastructure. The chapter also describes the key features and benefits provided by Avaya ACE, and how core communications capabilities are made available to IT developers through Avaya ACE Web services.

Chapter 2: An introduction to Web services and Parlay X

Chapter 2 provides a fairly high level introduction to Web services and the development of Web service clients, including descriptions of the different types of Web service that are provided by Avaya ACE, how the capabilities of the Web services are advertised and defined, and how to develop clients that can take advantage of these capabilities. The chapter also covers the Parlay X Web service interface specifications and how they are implemented by Avaya ACE.

Chapter 3: Avaya ACE real-time communications services

Chapter 3 describes the real-time communications capabilities that Avaya ACE Web services make available to client applications, including practical examples of how these capabilities can be employed to enhance business systems and processes.

Chapter 4: Avaya ACE information and system services

Chapter 4 describes the Avaya ACE Web services that enable client applications to obtain communications information and provide Avaya ACE system management capabilities.

Chapter 5: Developing Avaya ACE Web service clients

Chapter 5 describes the options available for accessing an Avaya ACE development configuration, how to obtain Avaya ACE Web service WSDL, how to generate a client proxy from the WSDL and finally how to develop a client application against the proxy. An example is used to demonstrate one possible approach to client development, using Apache Axis to auto generate a Java client proxy from Avaya ACE Web service WSDL.

Chapter 6: Web client development know-how

Chapter 6 works through a couple of real-life scenarios to provide a deeper understanding of the capabilities of some of the more functionally rich Web services, show how those Web services should be used and give practical hints and tips for developers.

Chapter 7: Avaya ACE packaged applications

Chapter 7 describes the functionality provided by each of the Avaya ACE Applications and add-ins, and looks in more detail at the Event Response Manager application and its programming interface.

Chapter 8: Next steps

Chapter 8 provides advice on next steps for developers who wish to learn more or who are keen to start communications enabling their business systems with capabilities provided by the Avaya ACE Web services.

Appendix A: Avaya ACE Web services and operations

Appendix A lists the available Avaya ACE Web services, together with the operations supported by each Web service and each operation's parameters.

Appendix B: Avaya DevConnect developer and partner program

Appendix B describes the benefits of joining the Avaya DevConnect program.

Appendix C: Comparison of programming interfaces

Appendix C provides a comparison of the capabilities and characteristics of the Avaya ACE Web services, the Avaya ACE Foundation Toolkit APIs and Avaya Aura Application Enablement Services.

References

The *References* section lists documentation and other resources referenced in the eBook.

Icons used in this eBook

The following icons are displayed in the margin to highlight notable information:



Important information about Avaya ACE Web services.



Future direction for Avaya ACE Web services application development: information about known and possible advances in future releases.



Helpful hints and additional information for Avaya ACE Web services application developers.

CHAPTER 1

ABOUT AVAYA ACE AND AVAYA ACE WEB SERVICES

In this first chapter of the eBook, you will discover what Avaya ACE is and how it enables developers to build applications that bridge the gap between an organization's business systems and its IP communications infrastructure. As you will learn, Avaya ACE allows business applications developers to incorporate communications functionality into business systems without having to acquire technical knowledge of the underlying communications network infrastructure or how it works. You will find out about the key features and benefits provided by Avaya ACE, and learn how core communications capabilities are made available to IT developers through Avaya ACE Web services.

At the end of the chapter we will also touch on some of the other features of Avaya ACE, including support for Unified Communications, deployment configurations and security.

What is Avaya ACE?

Avaya ACE is an open software platform for building Communications Enabled Business Systems (CEBS) and Unified Communications (UC) applications. Avaya ACE supports application enablement through a choice of two levels of programming interface, each bolstered with its own developer-friendly toolkit. In addition, Avaya ACE provides a set of plug-ins and packaged applications that offer, among other things, out-of-the-box integration into common desktop business applications, such as Microsoft Outlook, Internet Explorer, IBM Lotus Notes, Sametime and Microsoft Office Communications Server.

Application enablement interfaces and toolkits

Avaya ACE provides two toolkits that make it easy for application developers to create custom CEBS applications without having to deal with the complexities of the underlying network infrastructure.

Avaya ACE Developer Toolkit

The Avaya ACE Developer Toolkit exposes a set of Web services that enable access to communications capabilities provided by multiple vendors' communications platforms. The Web services provide a high level interface targeted at IT developers with little or no specialist communications knowledge.

Avaya ACE Foundation Toolkit

The Avaya ACE Foundation Toolkit includes a client-side Java API that enables access to Avaya Aura's communications capabilities, via the Foundation Runtime Services hosted on the Avaya

ACE server. The Foundation Toolkit API is a lower level interface than the Avaya ACE Web services, targeted at developers with some knowledge of Session Initiation Protocol (SIP). Avaya Aura Session Manager supports the routing of the originating and destination sides of SIP calls through a sequence of applications. These sequenced applications may be created using the Foundation Toolkit API, or be provided by Avaya Aura Communication Manager or Avaya CS 1000.

The Foundation Toolkit also supports the development of Endpoint Applications, which can act as the originating or destination endpoint of calls.

Important: the focus of this eBook is on the use of the Avaya ACE Web services. To learn more about the Foundation Toolkit API, see the *Avaya ACE Foundation Toolkit Overview*.^[8]

Avaya ACE middleware

As mentioned in the *Introduction* to this eBook, ACE stands for Agile Communication Environment. Using Avaya ACE, IT developers can endow business systems with communications capabilities, quickly and nimbly. In concrete terms, Avaya ACE is “middleware”: software deployed on a server that sits between, in this case, a business system and the communications network. The primary role that Avaya ACE performs in this position is to make functionality provided by the communications network available to the business systems through high level programming interfaces.

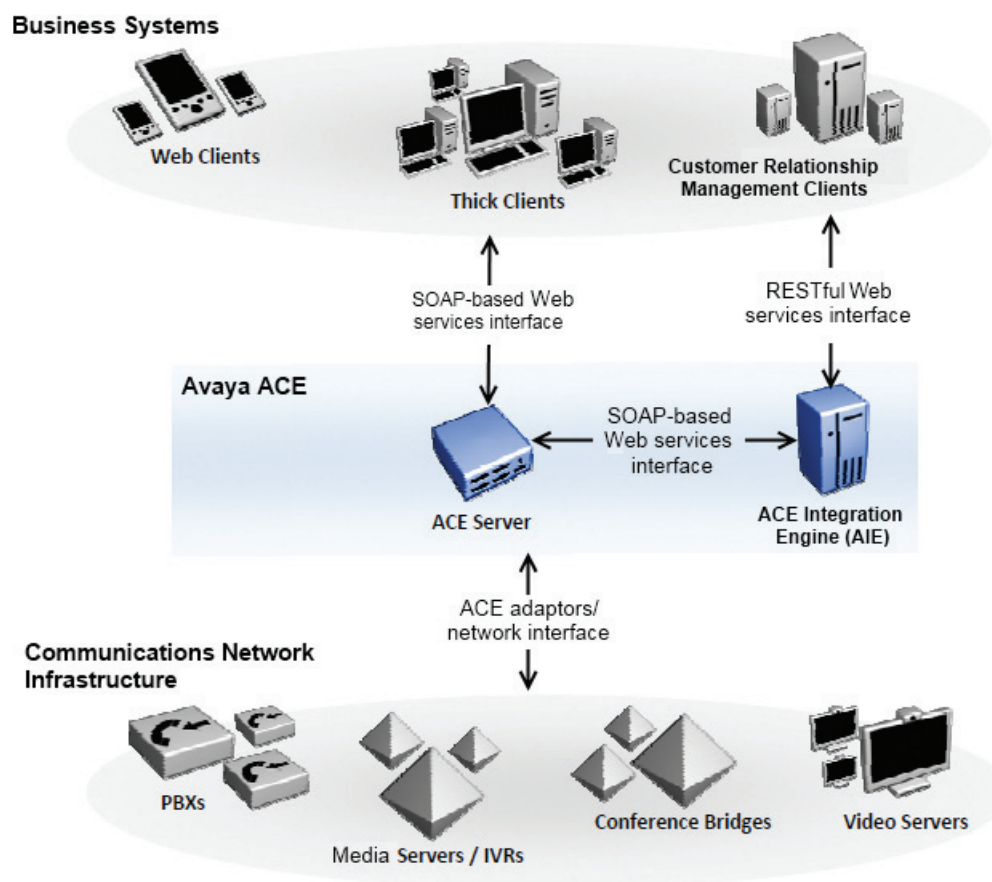


Figure 1.1: Avaya ACE sits between business systems and the communications network infrastructure

You'll notice from **Figure 1.1** that Avaya ACE comprises two server elements, the ACE Core Server and the ACE Integration Engine (AIE):

- **ACE Core Server:** hosts SOAP-based Web services that enable client applications to access core communications capabilities, and routes service requests to the underlying network elements that provide those capabilities. The ACE Core server also hosts the Foundation Runtime Services and some ACE applications.
- **ACE Integration Engine:** this is an optional ACE component that hosts a number of the Avaya ACE applications and provides a central point for administration of those applications. As mentioned previously, Avaya ACE offers a number of packaged applications that orchestrate the SOAP -based Web services hosted on the ACE Core server to provide additional, practical

communications capabilities, some of which are exposed to clients via simple RESTful Web services. We'll look the Avaya ACE applications in more detail in *Chapter 7*.

What differentiates Avaya ACE Web services from other CEBS interfaces?

The ability to communications-enable business systems isn't new. Many IP communications platforms expose programming interfaces that allow client applications to access their communications capabilities. A particular example is Avaya Aura™ Communication Manager which, when deployed as an Evolution server, exposes a variety of communications programming interfaces via Avaya Aura Application Enablement Services. However, using these more traditional interfaces can have a number of drawbacks:

- They typically provide a relatively low level abstraction of the underlying communications functionality, requiring the developer to have detailed knowledge of the platform, call control protocols, call flows and the interface itself.
- They are often proprietary interfaces or custom extensions to standards-based interfaces, and so can only be used to write client applications that work with one particular communications platform. As a result, when an organization upgrades its communications infrastructure, it often needs to reengineer its communications enabled business systems.
- They are usually programming language specific. For example, a couple of the more commonly used, standards-based interfaces are JTAPI (Java Telephony API), which is a Java interface, and TAPI (Telephony API), which is a Microsoft Windows-based interface. Therefore, if an existing business application is not written in a programming language that has a supported communications API, it becomes difficult or even impossible to incorporate communications functionality into that application.

In contrast, the Avaya ACE Web services:

- Provide a high level interface that can be used by IT developers with no specialist communications knowledge.
- Provide a common interface to multiple communications platforms from multiple vendors.
- Can be used to develop applications in just about any modern programming language.

Admittedly, use of low level interfaces is essential when developing applications that require a rich set of communications capabilities and a fine degree of control. However, where an enterprise applications developer simply wants to enhance business processes with basic real-time communications capabilities, using these low level interfaces is prohibitively complex. This is where

the Avaya ACE comes into its own: simplicity and flexibility are key reasons why Avaya ACE is the stand out environment for providing communications capabilities to business application developers.

Appendix C comprises a comparison of capabilities and characteristics of the Avaya ACE Web services, Avaya Aura Foundation Toolkit API and Avaya Aura Application Enablement Services.

Avaya ACE user profiles

In addition to enabling communications capabilities in business systems, Avaya ACE provides a number of other features that streamline and enhance communications within an enterprise. This is achieved primarily by maintaining a database of Avaya ACE users, containing information about the users, the communications media that are available to them, and their preferred contact devices. Client applications can use the information in the database to improve communications involving Avaya ACE users.

An Avaya ACE user can be:

- An individual end user, also known as a “subscriber”, who has been set up with a user profile on Avaya ACE.
- An entity that consumes Avaya ACE Web services, such as a packed Avaya ACE application or a third party client application.

The system administrator sets up user profiles for both types of user comprising, at a minimum, a user name and password for authentication. In addition, a user profile can be assigned to a user group, which determines the user’s access privileges and Web service access policies.

End users

An individual does not have to be set up as an Avaya ACE user to be included in calls, conferences and other communications created and maintained by Avaya ACE. However, having an Avaya ACE user profile does confer a number of advantages:

- The user profile can include personal information about the user, such as their full name, address and title, which can be accessed by client applications to display information about the user.
- The user profile can be assigned to a user group that determines the user’s access privileges. Depending on the assigned user group’s profile, the user can either access the full range system and user administration functions, manage user profiles that they have created themselves, or just manage their own user profile.

- The user profile can list the addresses of all of the user's contact devices, and define the order of preference in which those devices should be used to contact the user, for each type of supported media. Many of the Avaya ACE Web services allow participants in calls, conferences, etc. to be identified by their ACE address, in the format "ace:<ace_user_name>". When this address format is used in a request, the Web service queries the ACE user profile to determine the address of the device that should be used to contact the user. The actual device used is based on the user's preference settings and, in more sophisticated applications, the presence status of the user's devices obtained through the Avaya ACE Presence Web service (see *Chapter 2* for information about the Presence Web service).

Users can maintain their own profile, including contact devices and their preference settings, via the packaged Personal Assistant ACE application (see *Chapter 7* for information about Avaya ACE applications) or via a third party client application that makes use of the Avaya ACE User Profile Web service (see *Chapter 3* for information about the User Profile Web service) .

Typically, the employees of an organization that implements Avaya ACE will be set up as Avaya ACE end users. However, this does not mean that only Avaya ACE users can be participants in communications that are created and controlled by Avaya ACE clients. Client applications can make and manage calls, create conferences, etc. using the actual addresses of specific devices (typically tel URI or SIP format addresses for calls and conferences, but other formats can be used for different media devices – see *Chapter 2* for more details), rather than ACE addresses. This means, for example, that client applications can use the Avaya ACE Web services to make and manage calls to, and receive calls from, customers and other individuals outside of the organization.

Client application users

Each client that consumes Avaya ACE Web services, including packaged Avaya ACE applications and third party client applications, must be set up as an ACE user with a user name and password. The user profile must additionally be assigned to a user group with a policy that permits the required access (read only or read-write) to the Avaya ACE Web services and Web service operations that the client application consumes. All requests to access Avaya ACE Web service capabilities must include the client application's user credentials so that they can be authorized by the Web service before the requests are processed. The example client application in *Chapter 6* demonstrates how user credentials are passed to Web services.

Avaya ACE Web services

Avaya ACE avoids many of the drawbacks of using traditional communications programming interfaces by:

- Providing a simple programming interface that allows developers to access core capabilities that satisfy most of the communications requirements of business systems. Avaya ACE enables both real-time and non real-time communications capabilities, such as:
 - ◊ Making and controlling calls.
 - ◊ Creating ad hoc conferences and managing participants.
 - ◊ Playing and recording audio messages at communications devices.
 - ◊ Accessing and managing presence information.
 - ◊ Managing call forwarding rules.

In *Chapters 3 and 4* we'll look at these and other communications capabilities in more detail.

- Providing access to communications capabilities via Web services APIs. The use of Web services has a number of advantages:
 - ◊ Web services are a widely used technology, well known to most business developers.
 - ◊ Web services are programming language neutral. Client applications interact with the ACE Web services via SOAP messages sent over HTTP. Therefore, ACE Web service clients can be written in any language that supports the creation SOAP documents and their transmission over HTTP. This includes most of the commonly used programming and scripting languages. In fact, all of the modern programming languages, such as .Net, Ruby, Python, Java, C#, PHP and Ajax, provide tools and libraries that make it easy for applications to create, transmit and receive SOAP messages over HTTP.
 - ◊ Web service capabilities are described in standard Web Service Definition Language (WSDL) files. Avaya ACE publishes WSDL files for each of the Web service interfaces it provides. Standard developer tools are available that allow developers to generate application code automatically from the WSDL. For more information about SOAP, WSDL and Web service client development tools, see *Chapter 2*.
 - ◊ The Avaya ACE Web services are based on open standards. Most Avaya ACE Web services are Parlay X compliant.¹
- Providing programming interfaces that work with a wide variety of underlying communications network infrastructures provided by a variety of vendors.

It's worth looking at the last of these properties in more detail, as support for multiple network infrastructures from different vendors is one of Avaya ACE's key differentiators.

¹ Parlay X Web services APIs are defined jointly by the Parlay Group, the European Telecommunications Standards Institute (ETSI) and the Third Generation Partnership Program (3GPP). Parlay X compliant Web services are defined in standard WSDL files. Avaya ACE implements selected Parlay X version 2 and version 3 Web services APIs. See Chapter 2 for more information about the Parlay X specifications.

Avaya ACE applications and AIE RESTful Web services

ACE application interfaces

As we learned earlier in this chapter, the ACE Integration Engine (AIE) is an optional component on which selected packaged Avaya ACE applications are deployed. Avaya ACE applications orchestrate the SOAP-based Web services hosted on the ACE Core server to provide UC add-ins, business solutions and additional, high-level CEBS capabilities. Some of the Avaya ACE applications expose what are known as REpresentational State Transfer-compliant, or RESTful, Web services that allow developers to access their communications capabilities.

In *Chapter 2* you'll find out more about the use of RESTful Web services and how they compare to SOAP-based Web services.

Chapter 7 includes an overview of the available Avaya ACE applications. It also provides a more detailed description of the Event Response Manager application and, in particular, the use of its RESTful Web service API to trigger automated voice conferences when critical events occur in business systems.

Communications network infrastructure and vendor neutrality

As you learned earlier, Avaya ACE sits in the middle between an organization's business systems and its communications network infrastructure. Given that ACE is an Avaya product you might expect that the underlying infrastructure would have to be based on Avaya network elements. Indeed, Avaya ACE supports a range of Avaya platforms including Avaya Aura™ (with Communication Manager deployed as an Evolution server), Avaya Communication Server 1000, Avaya Communication Server 2100 and Avaya Multimedia Communication Server 5100. However, Avaya ACE also supports popular communications network elements from other vendors, including Cisco, Tandberg, IBM and Microsoft.

To achieve this network infrastructure and vendor neutrality, Avaya ACE implements what are known as "software adapters": one for each communications network it supports. As a developer, you don't need to use these adapter interfaces directly or be concerned with how they work, but they are a vital component of Avaya ACE because they are what allow the same set of Web services to access communications functionality provided by different networks.

Network infrastructure and vendor neutrality provides a number of key benefits:

- The same business application can be deployed in different environments with different underlying communications network infrastructures.
- Organizations can change their network infrastructure without having to rewrite their communications enabled business systems. For example, when an organization upgrades from Cisco Unified Communications Manager to Avaya Aura, applications that use Avaya ACE Web services capabilities supported on both platforms will continue to work with only a few updates to configuration settings.
- In an environment comprising multiple platforms from multiple vendors, Avaya ACE can be configured to use different network elements to provide different communications capabilities. Avaya ACE lets you configure rules and set the preferred service provider order to determine which network elements provide which services.

These features are unique to Avaya ACE and allow you to future-proof your business applications against changes to your communications network infrastructure.

Qualification

Note that the full set of Avaya ACE Web services is not supported equally in all communications infrastructure configurations. In addition, some of the Web services support only SIP or TR/87² configurations, although many can be used with either. See the *ACE Web Services* ^[1] guide for information about which capabilities are supported in which environments.

Some Avaya ACE capabilities require additional network elements to be available. For example, a supported Media Server is required to create an ad hoc conference or play an audio announcement in a call.

² SIP (Session Initiation Protocol) is an IETF-defined signaling protocol, widely used for creating, modifying, and terminating multimedia communications sessions with one or more participants. TR/87 is an ECMA specification that defines how CSTA (Computer-supported Telecommunications Applications) can be used over a SIP session to control and observe SIP user agents.

Other key features of Avaya ACE

Although the focus of this eBook is on communications enablement, Avaya ACE has a number of other notable features and benefits that are of interest to developers and their organizations.

Unified Communications capabilities

Avaya ACE provides Unified Communications (UC) capabilities through tight integration with popular third party communications systems such as IBM Lotus Sametime and Microsoft Office Communications Server (Microsoft OCS).

IBM Lotus Sametime integration

Avaya ACE offers a range of plug-ins that provide additional UC capabilities to Avaya ACE users through their IBM Lotus Sametime Connect client user interface, including:

- The ability to click on or select a contact to initiate a quick voice call. The called party does not have to be an Avaya ACE user.
- Avaya ACE users with a video device configured in their user profile can click on or select a contact to initiate a video call to another user. The called party must also have a video device configured in their ACE user profile.
- The ability to enter and call a number manually from a dialpad.
- The ability to initiate a conference call, and click on or select participants from a contacts list to add them to the conference.
- The ability to make a call to an Avaya ACE user's preferred contact device based on the preferences defined on the called party's Avaya ACE user profile.
- The ability to view aggregated telephony and/or video presence information for the user's contacts.

IBM Lotus Sametime can also integrate with the packaged Avaya ACE Hot Desking application. For more information about IBM Lotus Sametime Integration see the *Avaya Agile Communication Environment Administration – IBM Lotus Sametime Integration* guide. ^[2]

Microsoft OCS integration

Avaya ACE integrates with Microsoft Office Communications Server to provide Remote Call Control (RCC) and Presence services to Avaya ACE users via the Microsoft OCS client user interface. RCC capabilities include:

- The ability to click on a contact to make a call.
- Release an active call.
- Answer an incoming call.
- Redirect an incoming call to another device.
- View the calling party's details in a pop-up.
- Forward a call.
- Transfer a call without putting the calling party on hold.
- Place an existing call on hold and initiate another call to a consultant.

- Put a call on hold and subsequently retrieve it.
- View call information.

Microsoft OCS can also integrate with the packaged Avaya ACE Hot Desking and Personal Assistant applications. For more information about Microsoft OCS integration see the *Avaya Agile Communication Environment Administration – Microsoft Office Communications Server Integration* guide. ^[3]

Federated and high availability configurations

Avaya ACE supports a multi-regional deployment architecture, where multiple Avaya ACE servers are synchronized to form a logical group, or federation. This allows a user in one region to access call capabilities provided by a server located in another region.

In addition, Avaya ACE supports standalone and high availability (HA) configurations. In an HA configuration, Avaya ACE software is installed on active and standby servers. If the active server fails, processing automatically switches to the standby server.

Security features

Avaya ACE provides layered security features, protecting Avaya ACE solutions at all levels:

Server-side security: Avaya ACE uses the Secure Socket Layer (SSL) protocol to provide server-side authentication and ensure secure communications.

Client authentication and authorization: Access to Avaya ACE is controlled using HTTP basic authentication. All requests from client applications must contain a valid user name and password that corresponds to a user profile configured on Avaya ACE. The client application's user profile must have the appropriate access control rules set to invoke a particular Web service.

Web service communication is supported on secure (HTTPS) and non-secure (HTTP) ports. Client applications can choose to secure all Web services communications.

Round-up of Chapter 1

In this chapter you have learned what Avaya ACE is and how it makes communications capabilities available to business systems developers through the Avaya ACE Web services. You are now aware of Avaya ACE's key differentiators, including its ease of use, support for multiple programming languages and support for multiple network infrastructures.

Now you have got the Avaya ACE basics, we'll go on to learn more about Web services in general and about how the Parlay X Web service specifications have been implemented by Avaya ACE.

CHAPTER 2

AN INTRODUCTION TO WEB SERVICES AND PARLAY X

This chapter comprises a fairly high level introduction to Web services and the development of Web service clients. In it, you'll learn what Web services are, the different types of Web service that are provided by Avaya ACE, how the capabilities of the Web services are advertised and defined, and how to develop clients that can take advantage of these capabilities. In particular, the chapter covers SOAP-based Web services and the Parlay X Web service interface specifications.

If you are already familiar with using SOAP-based Web services, you can probably skip this chapter and go straight to *Chapter 3* to find out about the communications capabilities provided by the Avaya ACE Web services. However, if you're new to Web service development, or want to know about Parlay X and how the Parlay X specifications have been applied in Avaya ACE, read on.

We'll start by covering some the basic concepts, such as what Web services are, how client applications communicate with them and how they advertize their capabilities to developers. Then we'll go on to look at some of the tools available to help you develop client applications that use SOAP-based Web services, and finally introduce the Parlay X Web service specifications and find out how these have applied to the Avaya ACE Web services.

What are Web services?

Web services are self-contained, self-describing, platform independent applications that can be invoked from different environments across a network, including a company intranet or the World Wide Web. Let's look at what the various elements of this definition mean:

- **Self-contained.** Clients need no additional software to invoke a Web service; they simply need to be capable of generating and handling HTTP and XML messages. Each Web service is stand-alone and provides a discreet set of capabilities; there is no dependence on the state of the client application when requests are made to the Web service and, in that sense, are they are stateless. Multiple client applications can use the capabilities provided by the same Web service, without interfering with each other and without affecting the behavior of the Web service.
- **Self-describing.** To communicate, a Web service and its client must be able to recognize the format and content of the messages they exchange. To do this, the creator of the Web service provides service definition documents that describe its capabilities and how they can be accessed. Formatting information is carried in the messages themselves.

- **Platform independent.** Web services are based on open HTTP and XML standards that support interoperability between clients and the Web services across most modern computing platforms and programming languages.

Types of Web service

As mentioned above, one common feature of all Web services is the use of Hypertext Transfer Protocol (HTTP) to exchange messages with its clients. However, the two main types of Web service use HTTP in different ways:

- **SOAP-based** Web services (also known as “Big Web Services”) are accessed using XML messages that comply to the SOAP standard. SOAP messages are transmitted in the body of HTTP messages. SOAP originally stood for Simple Object Access Protocol. Although this description is no longer strictly accurate, the SOAP acronym continues to be used in its own right.
- **REpresentational State Transfer**, or RESTful, Web services are accessed using basic HTTP messages and standard HTTP operations.

So what are the differences between these two types of Web service? Let’s use a simple example to explain:

Imagine that an organization has two separate client applications that allow employees to create and maintain customer records. One of the client applications uses functionality provided by a SOAP-based Web service to update the customer database, the other client application uses a RESTful Web service to do exactly the same things.

An existing customer contacts the organization to inform them that she was recently married and has changed her name from Ms Jane Smith to Ms Jane Brown. Let’s compare how the request to update the customer’s family name would be processed by each client application:

SOAP-based Web service client: the client application makes a Remote Procedure Call (RPC) to the Web service to update the customer’s family name. To do this, the client creates a SOAP-encoded RPC message (a SOAP message) that it puts inside an HTTP request envelope and sends to the Web service URI using the HTTP POST method. The body of the SOAP message includes information about the service operation to be called (something like `SetFamilyName`) and its parameters, which might be the customer’s unique identifier (“12345678”) and her new family name (“Brown”). **Figure 2.1** shows what the HTTP request would look like, with the SOAP message in the HTTP message body. The Web service processes the request and, if successful, returns an appropriate HTTP response to the client, which may itself contain a SOAP message.


```

POST /UpdateCustomerDetails HTTP/1.1
Host: www.mycompanyx.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 362

<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap=http://www.w3.org/2001/12/soap-envelope
  soap:encodingStyle=http://www.w3.org/2001/12/soap-encoding>

  <soap:Body xmlns:m=http://www.mycompanyx.org/customer>
    <m:SetFamilyName>
      <m:CustomerId>12345678</m:CustomerId>
      <m:FamilyName>Brown</m:FamilyName>
    </m:SetFamilyName>
  </soap:Body>

</soap:Envelope>

```

Figure 2.1: A SOAP request in the body of an HTTP message

RESTful Web service client: each resource that can be accessed by a RESTful Web service client has its own URI. In this case, the customer's family name is a resource that can be accessed and therefore it has its own URI with a format something like:

`http://www.mycompanyx.org/rest/customer/12345678/familyName`

where "12345678" is Ms Jane Brown's (nee Smith's) unique Customer Identifier. To update the family name, the client application creates an HTTP request that includes a representation of the family name in the body of the message, and sends it to the resource URI using the HTTP PUT method. The PUT method adds the family name if it is not already there, or overwrites it if it is. In this case, the current value "Smith" is overwritten with the value "Brown". The representation of the family name in the HTTP message could be XML, JSON, HTML, or something else - it depends on what the Web service is designed to accept. **Figure 2.2** shows the HTTP PUT request with the family name represented using JSON.

```
PUT http://www.mycompanyx.org/rest/customer/12345678/familyName HTTP/1.1
Content-Type: application/json; charset=utf-8
Content-Length: 22

{"familyName":"Brown"}
```

Figure 2.2: A REST request in the body of an HTTP message

Obviously, this simple example doesn't give the full picture, but it does illustrate some of the key differences between the two types of Web service:

- Each SOAP-based Web service interface has a single URI to which all client requests are sent; a RESTful Web service has multiple URIs, one for each resource that can be accessed by the client.
- In SOAP-based Web services, operation calls are represented in the body of the SOAP message. Each Web service supports its own set of operations that need to be defined and made available to the client developer. RESTful Web services use standard HTTP methods only, namely:
 - PUT – to create or overwrite resources
 - GET – to retrieve resources
 - HEAD – to retrieve metadata about resources
 - DELETE – to delete resources
 - POST – to append resources to an existing set of resources
- SOAP-based Web services identify the data to be acted on in the SOAP message, usually as arguments of the operation calls; RESTful Web services identify the data to be acted on by their resource URIs.

As you can see, RESTful Web services are relatively lightweight: there's no need to document the API because it is standard for all Web services, and all messages are exchanged using simple HTTP requests and responses. SOAP-based Web services are "Big Web Services" because each has its own API that has to be documented by the Web service developer and understood by the client developer. They're also "big" in the sense that the messages passed between the Web services and their clients have a relatively large payload. Admittedly, that's not the whole story: to be able to use a RESTful Web service the client developer does need to know the URI of each resource and how those resources are represented. However, in most scenarios, RESTful Web services are considered easier to use and many purists believe that they are the only true "Web" services in the sense that they rely only on the basic technologies that support the World Wide Web.

The Avaya ACE Web Services

The core Avaya ACE Web services described in this eBook are all SOAP-based. So, you are probably wondering why RESTful Web services were introduced into the discussion at all. Well, there are a couple of reasons:

1. Some of the packaged ACE applications that are deployed on the ACE Integration Engine (AIE) expose RESTful APIs. For example, the Event Response Manager exposes a RESTful API that can be used to trigger automated conferences in response to monitored business events – see *Chapter 7* for more information about ACE applications and their interfaces.
2. Future releases of Avaya ACE may expose core communications functionality through a set of RESTful Web services hosted on the AIE. So, as a developer, you will be able to choose which type of Web service you want to use.



The RESTful Web services are not covered in this edition of the eBook. This edition focuses on how you can use the SOAP-based Web services in your client applications.

Web service definition files

Most SOAP-based Web services publish Web service definition files that provide all the information client application developers need to be able to use those services. The full definition of a Web service can comprise up to four different types of document:

- **Data Type Definition documents** define the types of data that can be exchanged between a client and the Web service, and the data's permitted values. Generic data types, such as “string” and date formats, are defined by the 3WC organization. However, if a Web service, or a set of Web services, uses non-generic or custom data types and value ranges, they are defined in Data Type Definition documents. XML Schema are used to define valid data types, and therefore Data Type Definition documents have an XSD extension.
- **Shared Faults documents** define common faults that might be returned if things go wrong when requests are made to the Web service, and which therefore need to be handled on the client side. In fact, the Avaya ACE Web services return two main types of exception: Service Faults and Policy Faults. Shared faults are defined in Web Service Definition Language, or WSDL, files with a WSDL extension – for all Avaya ACE Web services the Shared Faults Document is `parlayx_common_faults_2_0.wsdl` or `parlayx_common_faults_3_0.wsdl` depending on which version of a Web service you are using. We'll look Avaya ACE Web service fault handling in more detail in *Chapter 5*.

- **Service Interface documents** define a Web service interface, including the available operations plus the request, response and fault messages that are transmitted for each operation. Each Service Interface document can reference Data Type Definition and Shared Faults documents. Service Interface documents are WSDL files. A Web service can expose multiple interfaces and therefore can have more than one Service Interface document.
- **Service Binding documents** contain the binding to be used and the service definition associated with the binding. The binding defines the protocol (e.g. SOAP over HTTP) and operational style (e.g. RPC) to be used for interacting with the network. Each Service Binding document references one Service Interface document. Service Binding documents are also WSDL files.

Data types and faults are often common across multiple Web services, and therefore the same Data Type Definitions and Shared Faults documents can be referenced by multiple Web services. In practice, the Service Interface, Service Binding and Data Type Definitions can be combined in single WSDL file: most of the ACE Web services provide them as separate documents that reference each other, but other Web services, such as the Call Forwarding and Call History, combine them.

Although the full definition of a Web service might comprise both XSD and WSDL files, it is often referred to collectively as the Web service's WSDL.

Each of the core Avaya ACE Web services publish WSDL that clients can use to generate and send SOAP requests that the Web service recognizes, and that allow clients to handle responses and faults from the Web service. In fact, it is probably more precise to describe the core Avaya ACE Web services as being “WSDL-based”, but we’ll stick with our original description, “SOAP-based”, throughout this eBook.

WSDL is valid and well formed XML. **Figure 2.3** shows the first part of the interface WSDL for the Avaya ACE Third Party Call Control v3.4 Web service. Line numbers have been added to make it easier to reference some of the features mentioned earlier:

You can find the definition of the `makeCallSession` request starting on line 28 `makeCallSession` is one of the core operations supported by the Avaya ACE Web services, allowing client applications to programmatically initiate a voice call. The `makeCallSession` operation has five arguments, defined on lines 31 to 43:

- `callParticipants`
- `callingParticipantName`
- `charging`
- `mediaInfo`
- `reference`

Each argument has three attributes in addition to its name: `minOccurs`, `maxOccurs`, and `type`. Notice that `callParticipants` has a `minOccurs` value of “1”, whereas the other arguments have “`minOccurs=0`”: this indicates that `callParticipants` is the only mandatory argument, because at least one call participant has to be passed to the operation. `callParticipants` has a `maxOccurs` value of “2”, indicating that you can pass up to two call participants to the operation to set up a one or two party call, as required.

The `type` attribute defines the argument’s data type. In the case of the `callParticipants` argument each call participant is identified by their Avaya ACE URI or the URI of a specific communications device, so the `type` is “`xsd:anyURI`”. The `xsd` prefix corresponds to a namespace, which is defined on line 08 as “`http://www.w3.org/2001/XMLSchema`”. This indicates that `anyURI` is a generic Schema XML data type. If you go to the W3C web site, you’ll find the formal definition of `anyURI`, which is:

anyURI represents a Uniform Resource Identifier Reference (URI). An **anyURI** value can be absolute or relative, and may have an optional fragment identifier (i.e., it may be a URI Reference). This type should be used to specify the intention that the value fulfills the role of a URI as defined by [\[RFC 2396\]](#), as amended by [\[RFC 2732\]](#).

Other data types used by this and other Parlay X compliant Web services are defined in the `parlayx_common_types_3_1.xsd` Data Type Definitions document. The import statement for this file is on line 25.

The message that a client can expect to receive in response to `makeCallSession` is defined starting on line 46: `makeCallSessionResponse`.

```

01 <?xml version="1.0" encoding="UTF-8"?>
02 <wsdl:definitions
03   name="parlayx_third_party_call_interface"
04   targetNamespace="http://www.csapi.org/wsdl/parlayx/
05     third_party_call/v3_4/interface"
06   xmlns="http://schemas.xmlsoap.org/wsdl/"
07   xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
08   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
09   xmlns:parlayx_third_party_call="http://www.csapi.org/wsdl/parlayx/
10     third_party_call/v3_4/interface"
11   xmlns:parlayx_common_faults="http://www.csapi.org/wsdl/parlayx/
12     common/v3_0/faults"
13   xmlns:parlayx_common_xsd="http://www.csapi.org/schema/parlayx/common/v3_1"
14   xmlns:parlayx_third_party_call_local_xsd="http://www.csapi.org/schema/parlayx/
15     third_party_call/v3_4/local">
16
17   <wsdl:import namespace="http://www.csapi.org/wsdl/parlayx/common/v3_0/faults"
18     location="parlayx_common_faults_3_0.wsdl"/>
19
20   <wsdl:types>
21     <xsd:schema elementFormDefault="qualified"
22       xmlns:xsd=http://www.w3.org/2001/XMLSchema
23       targetNamespace="http://www.csapi.org/schema/parlayx/
24         third_party_call/v3_4/local">
25       <xsd:import namespace="http://www.csapi.org/schema/parlayx/
26         common/v3_1" schemaLocation="parlayx_common_types_3_1.xsd"/>
27
28       <xsd:element name="makeCallSession"
29         type="parlayx_third_party_call_local_xsd:makeCallSession"/>
30       <xsd:complexType name="makeCallSession">
31         <xsd:sequence>
32           <xsd:element name="callParticipants" type="xsd:anyURI" minOccurs="1"
33             maxOccurs="2"/>
34           <xsd:element name="callingParticipantName" type="xsd:string"
35             minOccurs="0" maxOccurs="1"/>
36           <xsd:element name="charging" type="parlayx_common_xsd:ChargingInformation"
37             minOccurs="0" maxOccurs="1"/>
38           <xsd:element name="mediaInfo" type="parlayx_common_xsd:MediaInfo"
39             minOccurs="0" maxOccurs="unbounded"/>
40           <xsd:element name="changeMediaNotAllowed" type="xsd:boolean"
41             minOccurs="0" maxOccurs="1"/>
42           <xsd:element name="reference" type="parlayx_common_xsd:SimpleReference"
43             minOccurs="0" maxOccurs="1"/>
44         </xsd:sequence>
45       </xsd:complexType>
46
47       <xsd:element name="makeCallSessionResponse"
48         type="parlayx_third_party_call_local_xsd:makeCallSessionResponse"/>
49       <xsd:complexType name="makeCallSessionResponse">
50         <xsd:sequence>
51           <xsd:element name="result" type="xsd:string"/>
52         </xsd:sequence>
53       </xsd:complexType>

```

Figure 2.3: Avaya ACE Third Party Call Control Version 3 WSDL

As you can see in **Figure 2.3**, it is possible for a developer to sight read a WSDL file, work out the available methods, responses and possible errors, and then manually write code to generate and send the required SOAP messages to the Web service. However, that would be very arduous

and time-consuming, and would require a high degree of programming knowledge. But fear not: as stated earlier, WSDL is XML-based and is therefore machine-readable. A plethora of tools, packages, libraries and standard APIs are available for most modern programming languages that can process WSDL and do most of the client development work for you. For example, Web service client development tools and technologies can be used to:

- Generate proxy classes on the client-side, directly from imported WSDL. The proxy classes implement stubs of all the available operations that are supported and actually implemented by the Web service. When the client application makes a method call to one of these stubs, the proxy converts the request into a SOAP message and sends it over HTTP to the Web service. The request is processed by the Web service, which may return a response or exception to the proxy in the form of a SOAP message. The proxy converts the response from the Web service into an object that you can manipulate in your client code.
- Invoke a Web service at runtime through its WSDL description, without having to generate or use compile-time proxy classes and method stubs.

Some tools can also generate sample applications and GUI-based test clients directly from the WSDL. You can use these to exercise all of the capabilities of the Web service, allowing you to observe what it does and how it works, and providing you with sample code that you can clone and adapt in your own client applications.

Web services client development

This section lists and briefly describes some of the more commonly used tools and technologies that are available to help you develop Web service clients. The list is far from exhaustive, but gives you an idea of what is available today to help you develop client applications that take advantage of the capabilities provided by Web services, including the Avaya ACE Web services.

Apache Axis for Java and C++. Apache Axis comprises a Java and C++ implementation of a SOAP server, plus tools and APIs for generating and deploying Web services, generating client proxy classes and creating, deploying and testing sample client applications. *Chapter 5* includes a detailed example of how to use the Axis plug-in to the Eclipse IDE to generate Java proxy classes on the client side, directly from WSDL, and how to use those classes in your client code.

Apache Web Services Invocation Framework (WSIF) for Java. WSIF is a simple Java API for invoking Web services at runtime through their WSDL descriptions.

Visual Studio for .NET. The .NET WebClient class provides common methods for sending messages to Web services and receiving responses from those Web services, over HTTP. Visual Studio includes tools that allow you either to add a reference to a Web service by importing its WSDL into a Visual Studio project, or to generate a .NET proxy for the Web service and use it to access the Web service in your Windows-based applications.

SOAP::Lite for Perl. Perl is a general purpose scripting language, originally developed for text manipulation but now used for a wide variety of tasks including web development. Perl includes a collection of modules, called SOAP::Lite, that provide simple SOAP interfaces on the client and server side. Developers can use SOAP::Lite to generate Web service proxies that, when called, generate SOAP messages, send them to a Web service and await the response.

Ruby on Rails and Soap4r for Ruby. Ruby on Rails is an open source web application framework for the Ruby scripting language. Ruby on Rails includes libraries, packages and utility scripts that help developers create Web services and Web service clients. Soap4r is a Ruby package that can be used to generate Ruby client code from Web service WSDL. The generated files can subsequently be used in client applications. Soap4r is a pure Ruby implementation of the SOAP 1.1 standard. Soap4r uses the XSD4R library to provide conversion between common data types.

soapUI is an open source Web service testing tool for software developers. Developers can use soapUI to generate Java client code and test clients from Web service WSDL.

Parlay X

Parlay X is a set of open Web service interface specifications for fixed and mobile communications networks. The specifications are defined jointly by the European Telecommunications Standards Institute (ETSI), the Parlay X Group and the Third Generation Partnership Program (3GPP).

The Parlay X specifications are in twenty parts, each describing a set of related Web service interfaces. The following Parlay X specifications are currently implemented by Avaya ACE Web services:

- Part 1: Common, versions 2 and 3
- Part 2: Third Party Call, versions 2 and 3
- Part 3: Call Notification, versions 3.2 and 3.8
- Part 9: Terminal Location, version 3
- Part 11: Audio Call, version 3
- Part 12: Multimedia Conference, version 3
- Part 14: Presence, version 3

Avaya ACE also includes a number of custom Web services that provide additional capabilities not covered by the Parlay X specifications. However, wherever possible, the custom Web services use the Parlay X base standard naming conventions and reference types. See *Chapters 3 and 4* for detailed information about Avaya ACE's Parlay X compliant and custom Web services, and their capabilities.

The Parlay Group web site that you may see referenced in older Avaya ACE product documentation is no longer available. However, you can still access the formal specifications and Web service definition documents from the 3GPP site:

<http://www.3gpp.org/ftp/Specs/html-info/29-series.htm>

Select the appropriate TS 29.199-*nn* link for the specification you want, where *nn* is the Part number. Parlay X Web service client developers are strongly recommended to read the specifications listed above, particularly the Part 1 specification that describes the Parlay X architecture.

Parlay X compliant Web services

You might imagine that developers of Web services start by implementing the underlying functionality, then expose that functionality through Web service interfaces, and finally create or generate the WSDL that describes the interfaces. However, to comply with the Parlay X specifications, Avaya had to start with the standard WSDL and then develop Web services that implemented the Web service definitions

Parlay X specifies common aspects of the Web service interfaces, including:

- Name spaces
- Data type definitions
- Fault definitions
- Interface and binding WSDL

Making Web services Parlay X compliant has a number of advantages for client developers:

- The Parlay X specifications define open standards that have been road-tested and adopted across the industry. The standards are well documented and have wide acceptance. Developers may already have had experience of using Parlay X compliant Web services.
- The Parlay X Web service interfaces were optimized to make it easy to access key communications capabilities, without software developers having to acquire specialized communications programming knowledge.

- Because the interface is standard, the underlying implementation can change without affecting or breaking the client application. This means your client application is not bound to the Avaya ACE Web services for all time; it could use other Web services that implement the same versions of the Parlay X specifications. Conversely, if you already have clients that use Parlay X compliant Web services, it would be relatively easy to switch to using Avaya ACE.

Round-up of Chapter 2

In this chapter you've been given a flying tour of the world of Web services: what they are, how they are defined, how they communicate and how to develop client applications that can be enabled with capabilities provided by Web services. Finally, you were introduced to the Parlay X Web service interface specifications on which the Avaya ACE Web services are based. Hopefully, this has given you the background information you need to go on and learn about the capabilities made available by the Avaya ACE Web services, starting in *Chapter 3* with their real-time communications capabilities.

CHAPTER 3

AVAYA ACE REAL-TIME COMMUNICATIONS SERVICES

In this chapter, we are going to investigate the communications capabilities that Avaya ACE Web services make available to client applications, and look at some practical examples of how these capabilities can be employed to enhance business systems and processes. The main focus of the chapter is on those services that support real-time communications and enable business applications to:

- Make voice calls
- Control ongoing voice calls
- Create and control conferences
- Play prerecorded in-call announcements at communications devices
- Get, manage and use presence information
- Set up and manage call forwarding rules at devices

As mentioned in *Chapter 1*, the capabilities available to client applications depend partially on which Avaya ACE Web services are implemented on the underlying communications infrastructure. To find out which capabilities are supported in the environment you are developing for, see the *Avaya ACE Web Services* guide. ^[1]

See also *Appendix A* for a list of Avaya ACE Web services and supported operations.

Making voice calls

Perhaps the most fundamental and widely employed real-time communications capability supported by the Avaya ACE Web services is the ability to make a two party voice call. This is the basis of what is often referred to as “third party call control”, where the client application is the “third party” that remotely initiates and controls a call between a calling party (the “first party”) and a called party (the “second party”). The application just needs the contact addresses of the first and second parties and to pass them into a simple “makeCall” Web service operation. The Web service rings the calling party; when the calling party answers the call the Web service rings the called party and puts both parties into a call.

So, business applications can initiate telephone calls, but why would they want to and what would prompt them to do so? Let’s look at some of the possible triggers for making calls and examples of when they might be used in real-life situations:

- User triggers:** A call could be triggered by user input at a client application's user interface. For example, the user interface displays a directory listing of the employees within an organization. When a user selects or clicks on a directory entry, the application automatically makes a call from the user to the selected employee. Another example is a click-to-call application that converts telephone numbers displayed on a web page to links. When the user clicks a telephone link the application automatically makes a call from the user to that number.
- Event triggers:** A call could be automatically triggered by a business event. For example, an application that monitors inventory levels of items held by an organization initiates a call from the organization's buyer to the supplier of a particular item whenever the number in stock falls below a predefined threshold.
- Scheduled triggers:** A call could be automatically triggered at a scheduled date and time, or as a recurring event. For example, an application allows a hotel guest to request a wake-up call and automatically makes the call to the guest's room at the specified time.

Call participant addresses

As mentioned above, for an application to make a call it needs the addresses of the parties it wants to put into the call. In the world of IP telephony a call address is a Universal Resource Identifier (URI) and can take a number of different formats depending on the types of devices supported by the underlying communications infrastructure. Avaya ACE supports the following address formats for call participants:

- tel URI:** Has the format tel:<directory_number>, where <directory_number> is the call participant's unique public telephone number represented in E.164 notation (e.g. tel:+1-201-555-0123) or their private extension number.
- SIP URI:** Has the format sip:<name>@<domain>, where <name> is the call participant's name or number and <domain> is the domain name of their network (e.g. sip:john.smith@acme.com or +12015550123@acme.com).
- ACE URI:** Has the format ace:<ace-user-name>. This format can only be used for call participant's who have a configured user profile on the Avaya ACE database. The ACE URI format is required to take advantage of some special features provided by Avaya ACE, including call line identifier (CLI) support and prioritized call routing – see the *Avaya ACE Web Services* guide ^[1] for details.

SIP and tel URIs correspond to a single address. However, ACE URIs may map to multiple addresses, each configured as a separate contact on a user profile. The relative priority assigned to each contact determines which address is actually used to make or receive a call.

The address formats that can be used depend on a number of factors, such as: the media type of the contact device, which address formats are supported for the Web service, which address formats are supported by the underlying communications infrastructure, the translation rules provisioned on the ACE server and whether or not the call participant has an Avaya ACE user profile.

In many cases, the addresses are stored and made available to client applications in a valid format, so developers do not need to be overly concerned about them. However, as you will learn later, some Avaya ACE Web services operations require and return particular address formats while other Web services support additional formats for IBM Lotus Sametime (supported by Presence Web service) and Microsoft Office Communications Server (OCS) environments.

Avaya ACE Web services used to make calls

Avaya ACE provides two Web services that can be used to make basic two party calls. Each of the Web services implements a different version of the Parlay X Third Party Call Control specification:

- **Third Party Call Control Web service version 2:** Allows an application to make a call between two parties. The application can subsequently use the Web service to cancel or end the call and get information about the current status of the call (see *Controlling ongoing voice calls*, below).
- **Third Party Call Control Web service version 3:** As well as making two party calls, this version allows an application to establish a communication session between a single user and a media resource so that it can, for example, play a prerecorded audio announcement to the called party using the Audio Call Web service (see *Playing prerecorded announcements* below).

An application can also use this Web service to add participants to a two party call that it originally initiated, thereby creating an ad hoc conference (see *Creating and controlling conferences*, below).



The version of the Third Party Call Control Web service you choose to use to make and control calls depends on a number of factors. If you want to take advantage of the additional call control capabilities provided by the Third Party Call Control Extensions Web service, you must use **version 2**. If you want to create single party calls or have the ability to create ad hoc conferences, you must use **version 3**.

Controlling ongoing voice calls

The Avaya ACE Web services allow applications to control various aspects of ongoing calls that have been initiated programmatically by the application itself or by some other means, such as one person manually dialing another. To be able to control a call, the application needs to know that a call has at least been initiated at an address and the call's unique identifier. There are two ways in which an application can get this information:

- When an application successfully initiates a call, a unique call identifier is returned to the application in the response from the Web service. The application can use the call identifier to subsequently access and control the call.
- An application can register to receive notifications of calls at specified addresses. Note that addresses must be identified by their tel URI; SIP and ACE URIs cannot be used for notifications. When a call is made to or from a monitored address, the application receives a notification that includes the call identifier, which it can then use to access and control the call. As you will learn later, applications can register to be notified of various in-call events as well.

Armed with the knowledge that a call is taking place, and with a key to accessing that call in the form of a unique call identifier, an application can control various aspects of the call, such as:

- Cancel the call before it is answered by the called party.
- Redirect the call to an alternative address before it is answered.
- Answer the call programmatically at the called party's address.
- Put the call on hold and subsequently retrieve it.
- Make a consultation call. This involves putting the called party on hold and making a call between the calling party and another party, such as a consultant, supervisor or subject expert. On completion of the consultation call, the called party can be transferred to the consultant or all three parties can be joined together in a conference call.
- Transfer the active call from the called party to another party in one step, without putting the calling party on hold. This is known as a "single step transfer".
- Generate Dual Tone Multi-Frequency (DTMF) digits at a device in the call. The application effectively simulates keypad presses (1 to 9, * and #) at the device to, for example, select a menu option or enter a security code.

- Get information about the call including its current status (in progress, active or terminated), its start time, its duration (if it has already ended) and the reason for it being ended. Note that call information only remains available for about 30 seconds after a call finishes.
- End the call.

Avaya ACE Web services used to control calls

Avaya ACE provides the following Web services that allow applications to control calls:

- **Third Party Call Control Web services v2 and v3:** allow an application to control and get information about calls that it either initiated itself or of which it was notified by one of the Call Notification Web services.
- **Third Party Call Extensions Web services:** Avaya ACE provides two versions of the Extensions Web service, which extend the capabilities of the Third Party Call Control Web service version 2. Although based on Parlay X principles, these Web services are not Parlay X compliant.

Version 2.3 allows an application to answer a call, create a consultation call, put a call participant on hold and subsequently retrieve the call, redirect an incoming call to another address, perform a single step transfer, and generate DTMF digits.

Version 2.0 provides similar functionality, but does not support single step transfers or the generation of DTMF digits.

The Extensions Web services can be used to control calls that were initiated using the Third Party Call Control Web service version 2 or of which the application was notified by one of the Call Notification Web services.



Version 2.3 of the Third Party Call Extensions Web service is recommended for new development.

- **Call Notification Web services:** Avaya ACE provides two versions of the Call Notification Web service, each of which implements a different version of the Parlay X Call Notification specification.

Version 3.8 allows applications to register to be notified when calls are made *to* or *from* specified addresses. Applications can also register to be notified of the following in-call events:

- called party has answered the call and the call is now active
- called party address busy
- no answer at called party address
- call has been disconnected after it was answered

When the telephone rings at one of the monitored addresses, and subsequently when one of the other monitored call events occurs, the application receives a notification message that includes the calling party's name (if known) and address, the called party's address and a unique call identifier. The identifier returned in event notifications can be used to access and control the call via the Third Party Call Control and Third Party Call Extension Web services.

Version 3.2 provides similar capabilities to **version 3.8**, but only allows applications to register to be notified about calls made *to* specified addresses. **Version 3.2** also uses a slightly different set of operations to notify call events.



Avaya recommends using **Version 3.8** of the Call Notification Web service for new development. **Version 3.8** provides all the capabilities of **version 3.2** and more, and will be able to notify additional call events if they are made available in future releases.

Creating and controlling conferences

Technically, a conference can have none, one, two or more participants, but for our purposes we will define a conference as being a call involving three or more parties.

The benefits of being able to create conferences programmatically, and the triggers that might be used to prompt an application to do so, are similar to those for making calls. However, one example worth singling out is the ability of an application to react to a critical business event by immediately putting all of the stakeholders into a conference so that they can discuss and resolve the issue with minimum delay. In fact, Avaya provides the packaged Event Response Manager ACE application which makes it even easier to support this scenario in your client applications – see *Chapter 7* for more information about the Event Response Manager.

Ad hoc conferences

As we discovered earlier in this chapter, the Third Party Call Control Web service version 3 can be used by an application to make a two party voice call and then to add participants to the call on an ad hoc basis. To add a participant to an ongoing call, the application just needs the call's unique identifier, which it has because it created the original call, and the new participant's address. The application then passes these values into the Web service's "addCallParticipant" operation.

The application can use the Web service to control the conference by transferring a participant to another call, removing a participant from the call, getting participant information and ending the conference.



The conferencing capabilities provided by the Third Party Call Control Web service version 3 are dependent on the availability of a supported Media Server, and on the Media Server being enabled on the underlying communications platform. See the *ACE Administration* guide ^[4] for details.

Avaya ACE Web services used to create and control conferences

Avaya ACE provides the following Web service that allows applications to create and control conferences:

- **Third Party Call Control Web service version 3:** used to create ad hoc conferences.

Playing prerecorded announcements



The ability to play prerecorded announcements is dependent on the availability of a supported Media Server, and on the Media Server being enabled on the underlying communications platform. See the *Avaya ACE Administration* guide ^[4] for details.

Avaya ACE allows client applications to play announcements to participants in ongoing calls and conferences that were originally created using the Third Party Call Control Web service version 3.

As you learned earlier, when an application creates a call using the Third Party Call Control Web service version 3 the call's unique identifier is returned to the application. The application can then use the identifier not only to control the call, including adding participants to turn the call into an ad hoc conference, but also to play prerecorded announcements and other forms of audio content to the participants in the call.

The ability to play audio announcements in calls and conferences has a wide variety of practical applications. For example, you may remember that the Third Party Call Control Web service version 3 allows an application to make a call to a single party. At first glance this would seem futile, but if the application adds a media resource to the call and plays a prerecorded announcement to the called party, the purpose of one party calls becomes apparent. Another possible use for audio announcements is when a call or conference is triggered by a business event. A standard message can be played to the participants describing the event: otherwise the participants wouldn't know why they had been called or what they were meant to be discussing.

To be able to play an announcement in an ongoing call or conference, an application needs to pass the call identifier and the location (URL) of the message file on the Media Server into the appropriate Web service operation. Alternatively, if a Media Resource Control Protocol (MRCP) server is configured in the environment, the application can pass a text message to the Web service which will be rendered as audible speech.

Once the application has initiated the playing of an announcement, it can:

- Check the current status of the announcement for each participant in the call, which may be pending, playing, played or error.
- Terminate the announcement.

Recording and Playing Announcements using the Message Drop and Blast Service

Message Drop and Blast is an optional Avaya ACE application that orchestrates underlying audio recording and call control Web services to enable automated voice recording and broadcast of audio messages to specified recipients. The application is deployed on the core ACE server.

In a typical deployment, the Message Drop and Blast application interfaces with a Customer Relationship Management (CRM) system. The Message Drop and Blast Web services provide capabilities that can be used directly by client applications to record announcements, play announcements in ongoing calls, and to call a list of recipients at a specified time to play an announcement to them.

For more information about the Message Drop and Blast Web services, see the *Avaya ACE Web Services* guide. ^[1]

Avaya ACE Web services used to play announcements

Avaya ACE provides the following Web services that allow applications to play prerecorded announcements:

- **Audio Call Web service:** a Parlay X compliant Web service that allows applications to play announcements in calls and conferences originally initiated by the Third Party Call Control Web service version 3.
- **Message Drop and Blast application Web services:** Message Drop and Blast is an ACE application that exposes a series of custom Web services. The Message Drop and Blast Web services enable client applications to record audio messages spoken at a specified device, play messages in calls, schedule messages to be played to a list of recipients and enable click-to-dial calling.

Managing and using presence information

Presence is the ability of a party to indicate their availability or willingness to be contacted on a particular device. In presence terms, the combination of a party and a device is known as a “presentity”. Applications and other clients can subscribe to a presentity and subsequently request the presentity’s current presence status and to be notified of any changes to the presentity’s presence status. In this role, the client application is known as a “watcher” of the presentity.

Applications can use a party's tel URI, SIP or ACE address, or IBM Lotus Sametime user ID, to request presence information. tel URI, SIP and IBM Lotus Sametime addresses correspond to a single address. ACE addresses can be used to request aggregated presence information from multiple addresses defined in the party's ACE user profile.

Business applications can use presence information in many different ways, for example:

- To display the current availability of parties in a contact list or directory.
- To provide a user interface that allows users to update their presence status.
- To check whether a party is available at a device before attempting to put them into a call or conference.

The Avaya ACE Web services provide a number of presence-related capabilities that allow applications to:

- Subscribe to one or more presentities. An application can only get current presence information, and be notified of changes to presence status, for presentities that it is subscribed to.
- Get the current presence status of a party at a specified device or aggregated across all of their devices. The request returns presence information for each device, including:
 - ◊ when its presence status last changed
 - ◊ the relative priority given to the device as a means of contacting the party (for aggregated presence only)
 - ◊ the means of communication at the device (for aggregated presence only)
 - ◊ the status of the device — on, off or busy
 - ◊ the party's current activity at the device - none, available, on the phone, do not disturb, etc.
- Request to be notified of changes to the presence status of one or more presentities. Whenever the presence status changes at a presentity, a message is sent to the application including presence information as described above. Applications can subsequently request to stop being notified of presence status updates.
- Dynamically update a presentity's presence status.
- Get a list of watchers of a specified presentity. The application can optionally request that the list be filtered by watcher status – authorized, blocked or polite blocked.

Avaya ACE Web services used to get and manage presence information

Avaya ACE provides the following Web service that allows applications to get and manage presentities' presence information:

- **Presence Web service:** a Parlay X compliant Web service that allows applications to subscribe to, get and request notification of a presentity's presence information. The Web service also allows applications to update presence information.

Setting up and managing call forwarding rules

Applications can set up rules so that calls made to a specified address are automatically and immediately forwarded to an alternative address. Applications can also get information about the call forwarding rules that are in place at an address and turn off call forwarding.

Avaya ACE Web services used to set up and manage call forwarding rules

Avaya ACE provides the following Web service that allows applications to set up and manage call forwarding rules:

- **Call Forwarding Web service:** a non-Parlay X Web service that allows applications to set up call forwarding rules at a specified address, get the current call forwarding rules at a specified address, and cancel the call forwarding rules in place at a specified address. Currently, the Web service only supports tel URIs.

Round-up of Chapter 3

In this chapter you learned about the capabilities of the Avaya ACE Web services that allow developers to endow business systems with real-time communications capabilities, quickly, easily and without the need to acquire in-depth IP communications programming knowledge. In the next chapter we will go on to look at the remaining Avaya ACE Web services that provide information services, such as call history and device location information, and ACE system services, such as system monitoring and ACE user profile management.

CHAPTER 4

AVAYA ACE INFORMATION AND SYSTEM SERVICES

The previous chapter described the Avaya ACE Web services that enable developers to enhance business systems with real-time communications capabilities. In this chapter, we are going to look at the remaining Avaya ACE Web services that enable client applications to obtain communications information and that provide Avaya ACE system management capabilities.

Information services allow developers to create clients that can:

- Get call history information
- Get information about the physical location of mobile communications devices

System services allow developers to create clients that can:

- Set up and manage Avaya ACE users and user groups
- Get Avaya ACE user information from local and remote regions in federated configurations
- Monitor the health of Avaya ACE systems

The information capabilities available to client applications depend partially on which Avaya ACE Web services are implemented on the underlying communications infrastructure. To find out which capabilities are supported in the environment you developing for, see the *Avaya ACE Web Services* guide. ^[1]

System capabilities do not depend on the underlying communication infrastructure and are therefore available in all Avaya ACE environments.

See *Appendix A* for a list of all Avaya ACE Web services and supported operations.

Information services

Getting call history information

Avaya ACE makes it possible to get historic information about calls involving Avaya ACE users. To start, the application registers to receive call information from one or more contact devices for a specified Avaya ACE user. The act of registering the devices prompts Avaya ACE to start collecting and saving incoming and/or outgoing call information at each of those devices. The application can subsequently request the call history at specified or all registered devices for an Avaya ACE user. The call history records returned by the request include the following information for each call:

- Direction – incoming or outgoing
- Calling party's name and address
- Called party's address
- Status – answered, not answered, forwarded or unknown
- When the call started and how long it lasted

Applications can also delete specified call history records and deregister devices for a specified user. Call history information can be used by applications which, for example, monitor and analyze contact center agent call activities or which make call history information available to users for informational purposes.

Call history capabilities are provided by the Call History Web service.

Getting information about the location of communications devices

Client applications can get information about the physical location of a specified communications device. This location information can be requested and returned as co-ordinates (latitude, longitude and altitude) or as the distance in meters from a specified point. Applications can also dynamically update location information for a device.

Location information can be used by applications that, for example, want to track mobile devices and provide information and services to those devices based on their geographical location.

Note that only device-specific tel URI and SIP address can be used in location requests; ACE addresses may correspond to multiple devices, so they cannot be used.

Device location capabilities are provided by the Parlay X compliant Terminal Location Web service and the Location Supplier Web service.

System services

Creating and managing Avaya ACE users, user groups and global security settings

Avaya ACE supports capabilities that enable client applications to manage Avaya ACE users and user groups, including:

- Adding and deleting users
- Adding, maintaining and deleting contact information on user profiles

- Updating personal details on user profiles
- Adding and deleting user groups
- Adding users to, and removing users from, user groups
- Maintaining buddy groups and their members
- Updating global security settings

These capabilities allow organizations to build their own Avaya ACE management interfaces, or to incorporate Avaya ACE management capabilities into existing systems, as an alternative to using the Avaya ACE GUI.

Avaya ACE user and user group management capabilities are provided by the User Profile Web service.

Getting Avaya ACE subscriber information

An Avaya ACE “subscriber” is an individual end user who is set up with an ACE user profile and contact details. Subscribers are distinct from other entities that may be set up with an ACE user profile, such as Web service clients and ACE applications.

In federated configurations, up to four Avaya ACE servers can be deployed in different geographical regions. Each ACE server stores subscriber information in its own LDAP database, which is searchable from all of the ACE servers in the federation. Subscriber information is a subset of the User Profile data, which includes the user’s ACE user name, personal and organizational details, contact information, and the IP address of their home ACE server.

Applications can request subscriber information about a specified Avaya ACE end user, by their user name or by the address of one of their communications devices. The search for the user can be limited to a specified Avaya ACE region or extend across all regions in a federated configuration. Requests can be to return all or selected information about a user.

In a federated configuration, the IP address of the user’s home ACE server returned by a search can be used to access the Web services hosted on that server to manage the user’s profile, monitor their presence status, etc.

Avaya ACE subscriber information capabilities are provided by the Subscriber Management Web service. Using the Web service, client applications can get information about ACE users in other regions and use that information to provide communications capabilities to those users.

Monitoring the health of Avaya ACE systems

Avaya ACE supports a capability that enables client applications to monitor the health of Avaya ACE servers. In particular, clients can perform checks to determine whether the Avaya ACE server has recently been restarted. A client may need to know that a server has restarted if, for example, it is using call history information. This is because call history information may not be recorded for calls made in the period immediately after a restart.

The client application periodically sends a request to Avaya ACE to obtain information about the length of time the Avaya ACE server has been up and running, and to get the restart token which is issued each time the server is restarted. If the restart token is different for consecutive requests, the server must have been restarted between times.

Avaya ACE health monitoring capabilities are provided by the System Monitoring Web service.

Round-up of Chapter 4

In this chapter you have learned about those capabilities of the Avaya ACE Web services that don't relate directly to the enablement of real-time communications: namely information and system services. Now that you are familiar the full range of capabilities enabled by Avaya ACE Web services, we'll go on to look at some of the practicalities of developing client applications that make use of these capabilities.

CHAPTER 5

DEVELOPING AVAYA ACE WEB SERVICES CLIENTS

Now that you are familiar with the capabilities of the Avaya ACE Web services, it's time to learn how you can take advantage of those capabilities to communications-enable your business systems. *Chapter 2* listed a number of popular tools and technologies that are available to help develop Web services clients. It was also mentioned that these tools use different methods to access Web services from their WSDL. Obviously, it is impractical to cover all the tools and the various approaches here, so we'll use one which makes it easy for you to see what is happening and learn how Web services clients work.

In this chapter, you will learn about the options for accessing an Avaya ACE development configuration, how to obtain Avaya ACE Web service WSDL, how to generate a client proxy from the WSDL and finally how to develop a client application against the proxy. An example is used to demonstrate one possible approach to client development, using Apache Axis to auto generate a Java client proxy from Avaya ACE Web service WSDL. The example is then used to explain what the proxy, interface and other generated classes are, before taking you step-by-step through the process of building a simple Java client.

Avaya ACE development configurations

To successfully integrate Avaya ACE Web services capabilities into your business systems, you will need access to a development configuration of ACE so that you can prototype, test, debug and demonstrate your client code. Developers without access to an Avaya ACE configuration within their own organization may be able to utilize the DevConnect Avaya ACE Remote Lab for many of their development and prototyping activities.

The Avaya ACE Remote Lab gives registered DevConnect members no-cost access to an Avaya ACE development configuration so that they can self-test solutions that include communications capabilities enabled by the Avaya ACE Web services. Members can book on-demand lab access for periods of three months at time. Additional information about the Avaya ACE Remote Lab is available from the DevConnect web portal.

Obtaining Avaya ACE WSDL

As we discussed earlier, to be able to use an Avaya ACE Web service you need to obtain the WSDL which defines what the Web service can do and the format of the request and response messages that it can process. So, from where do you get the Avaya ACE Web service WSDL?

If you have direct access to an Avaya ACE environment, the best place to obtain the WSDL is from the **Web Service WSDL Download** option under the **Help** menu on the Avaya ACE GUI, as shown in **Figure 5.1**.

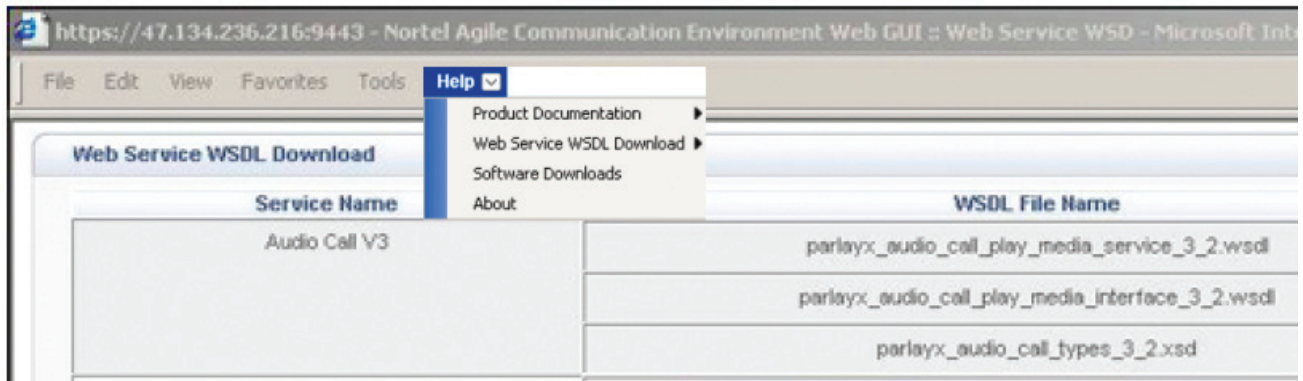


Figure 5.1: Downloading Web service WSDL from the Avaya ACE GUI

Alternatively, registered Avaya DevConnect members can download a ZIP file containing the complete set of the Avaya ACE Web service WSDL files from the DevConnect portal. ^[5]

Generating client code from WSDL

Using Apache Axis to generate a Java client proxy

This example demonstrates show you how to use the Apache Axis plug-in, running inside Eclipse, to generate a Java client proxy directly from a Web service's WSDL. The example uses Eclipse Java EE for Web Developers - Helios release, which includes the Apache Axis tools. The WSDL is for the Avaya ACE Third Party Call Control version 2.3 Web service. Just to forewarn you, the example assumes that you have some familiarity with Java and Eclipse, know how to open an Eclipse workspace and know how to create new Java Projects, Packages and Classes.

Start by opening a new workspace in Eclipse, then.

1. Create a new Java Project, named “MyACEWebServiceClient”.
2. Right click on the project in the Package Explorer, and select **New** then **Other ...** - the **New** dialog box is displayed.
3. Open the **Web Services** folder and select **Web Service Client**, as shown in **Figure 5.2**.

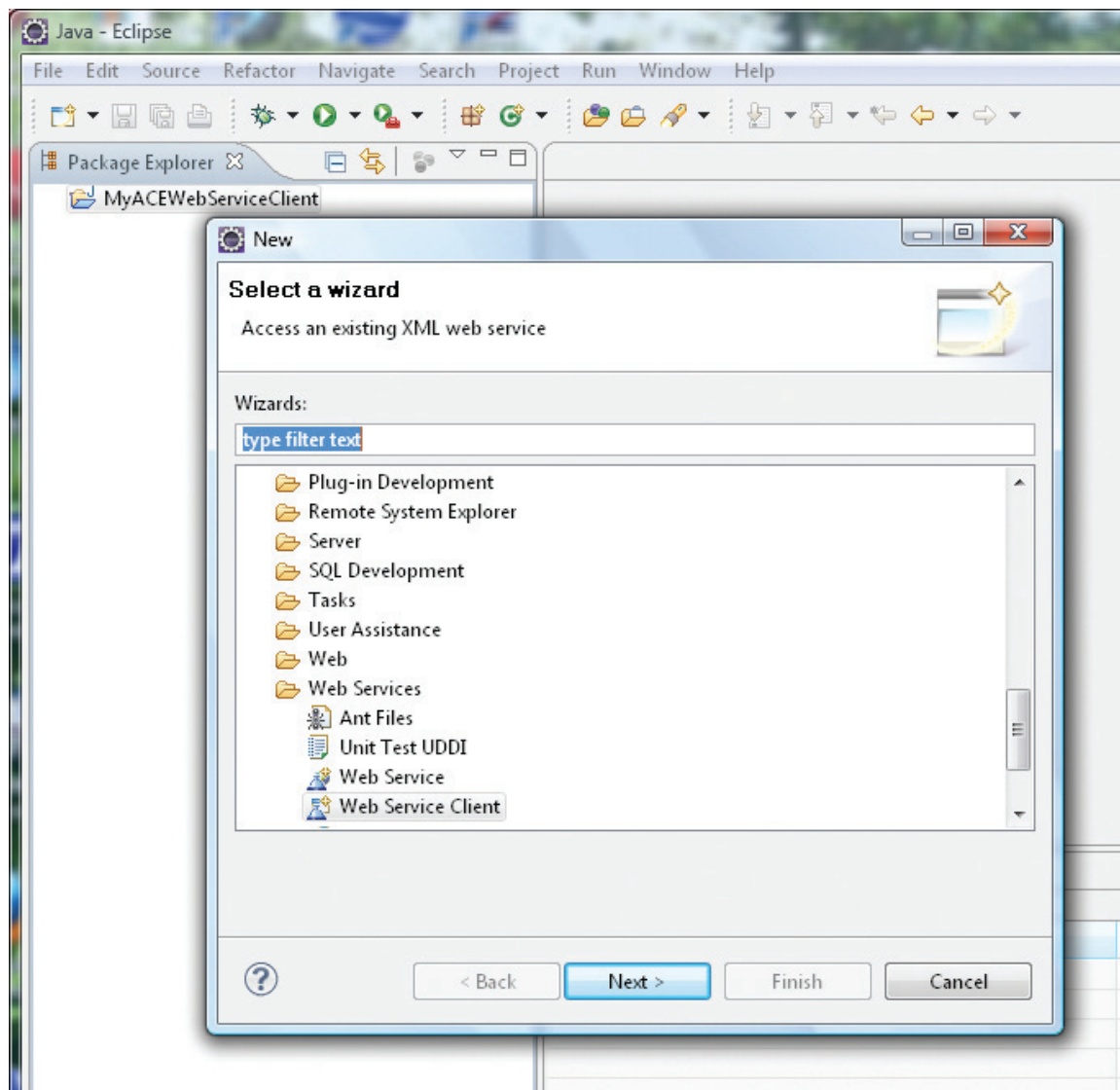


Figure 5.2: Select the Web Service Client Wizard

4. Click **Next** – the Web Services Client dialog box opens.
5. Set the slider on the left its lowest possible value, which is “Develop client”. This will only generate the client proxy. You can set the slider to additionally generate, assemble, deploy, install, start and test a sample client application. For this example, we’ll write our own client code to help understand what’s involved, so just create the client proxy.
6. Enter the location of your Avaya ACE Third Party Call Control v2.3 WSDL file in the **Service definition** field, as shown in **Figure 5.3**. Note that this is the Service Binding WSDL file that references the Service Interface, Data Type Definition and Shared Faults documents that define the Web service.

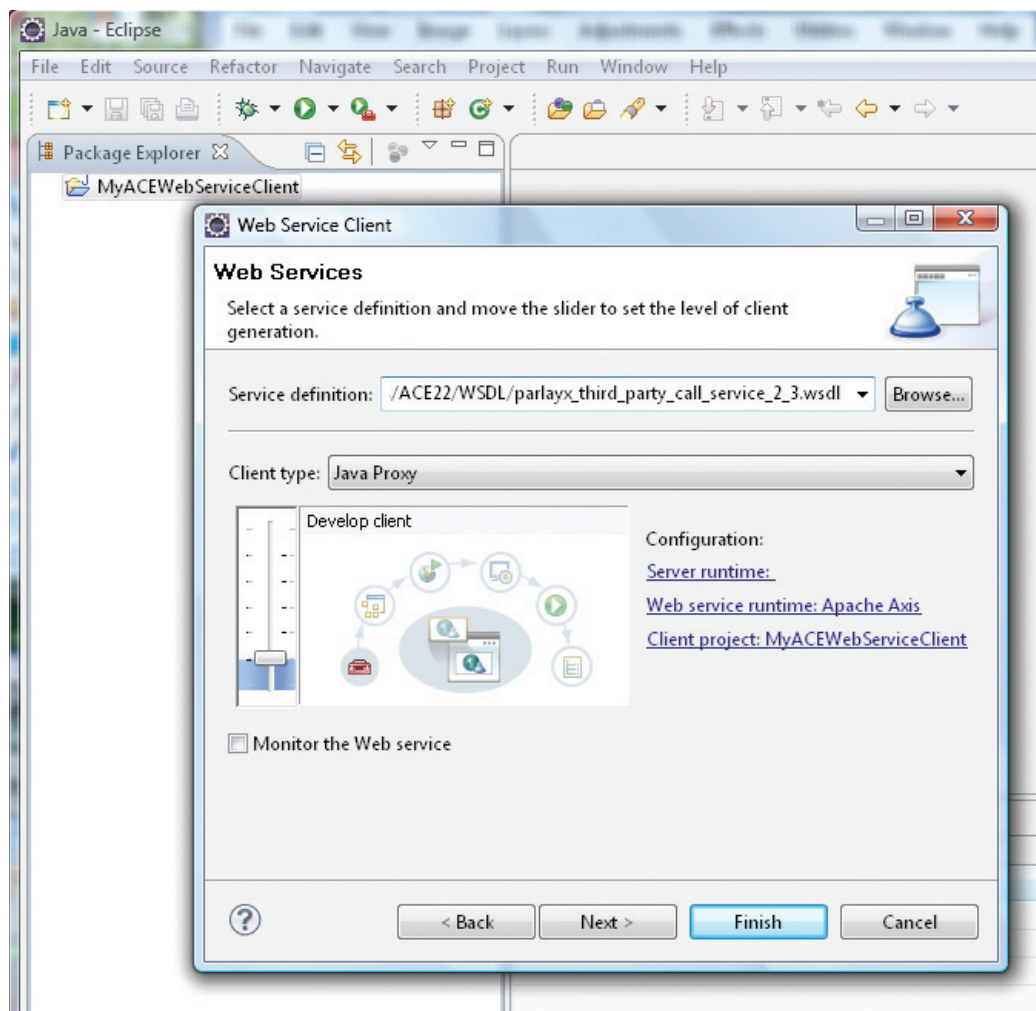


Figure 5.3: Import the Service Binding WSDL

7. Click **Finish** – the Apache Axis plug-in starts generating the Java client proxy class, proxy interface, and the other Java classes that the client needs to be able to use the Web service.

From start to finish, it takes just a couple of minutes to generate all the client-side classes you need to start developing your client application. But before we start writing any client code, let's take a look at the classes that were generated by Apache Axis from the WSDL.

Exploring the generated client proxy code

Figure 5.4 shows the four packages generated by Apache Axis. We'll explore each in turn, starting with the package that contains the main client proxy class and its interface, namely `org.csapi.www.wsdl.parlayx.third_party_call.v2_3._interface`.

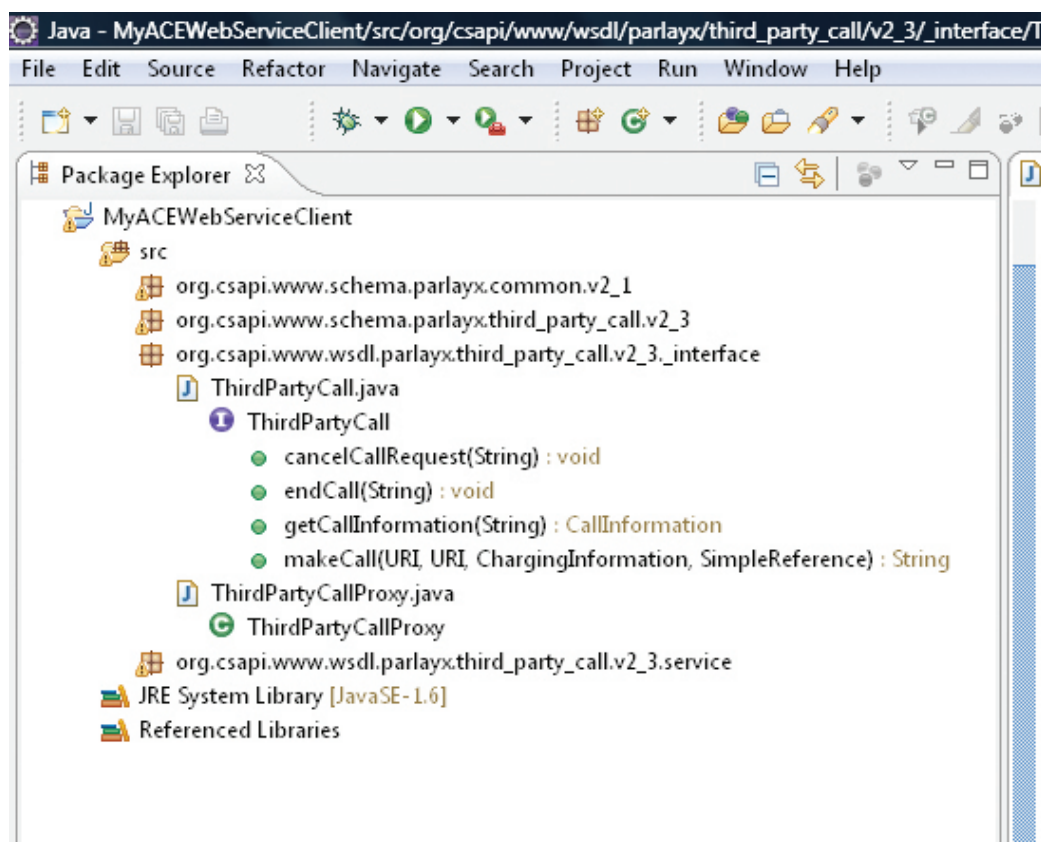


Figure 5.4: Generated Java packages, with interface package expanded

The main client proxy class, `ThirdPartyCallProxy`, implements the service interface `ThirdPartyCall`. As you can see, the service interface exposes four methods that correspond to the operations supported by the Web service: `makeCall()`, `getCallInformation()`,

`cancelCallRequest()` and `endCall()`. These are the service methods that you can call directly from your client application, as you will discover later.

When you successfully call one of these methods, the Web service may return an empty HTTP response, like `cancelCall()` or `endCall()`, or a SOAP message that is subsequently converted to a Java object on the client. `makeCall()` returns a simple `String` object, which actually contains the unique identifier assigned to the call, but `getCallInformation()` returns a more complex `CallInformation` object. The `CallInformation` class was also generated by Apache Axis, in the package shown in **Figure 5.5**.

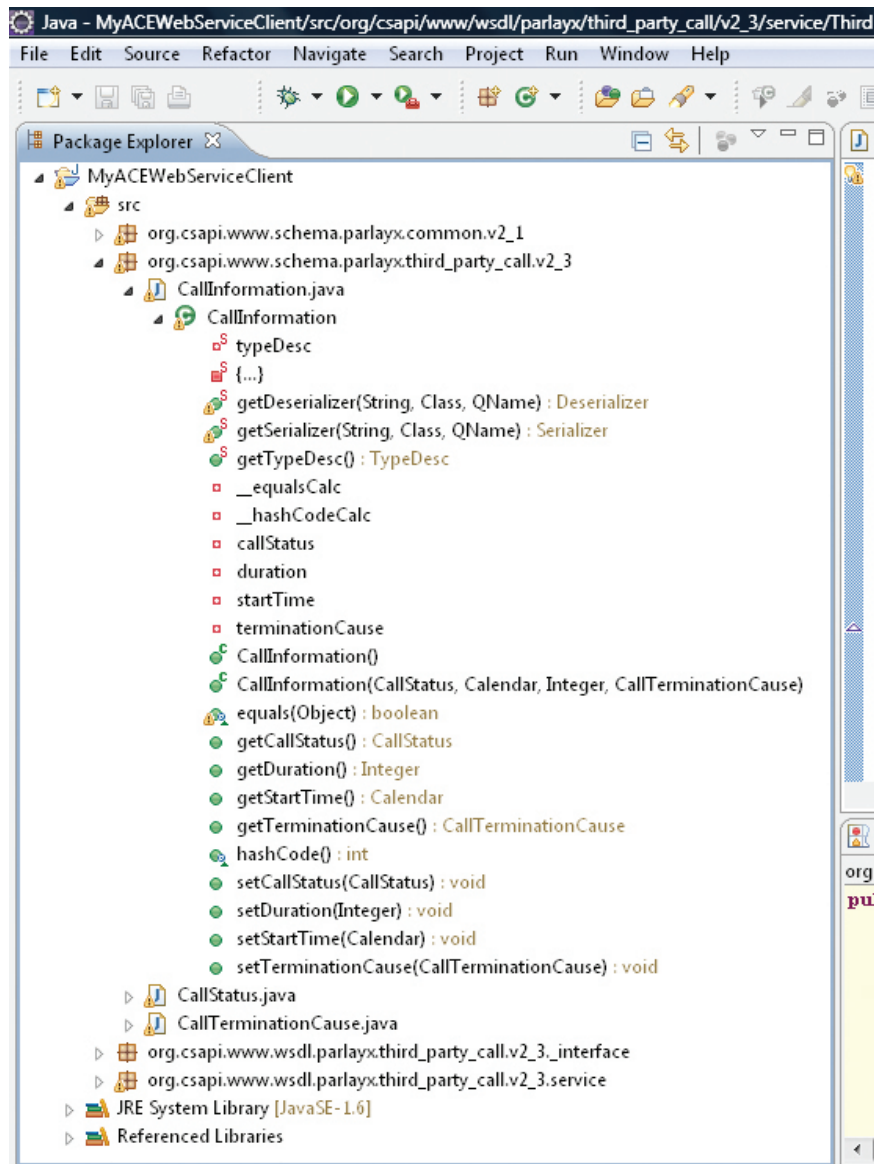


Figure 5.5: Third party call information classes

You can call a number of methods on a `CallInformation` object to get information about a call, such as when it started (`getStartTime()`) and its duration (`getDuration()`). Notice that two of these methods, `getCallStatus()` and `getTerminationCause()` return objects that are also defined in this package.

The `org.csapi.www.wsdl.parlayx.third_party_call.v2_3.service` package, shown in **Figure 5.6**, contains the classes that are used by a client to locate the actual Avaya ACE Third Party Call Control Web service on the Avaya ACE web server, and to create an instance of the service on the client-side, to which requests can be made.

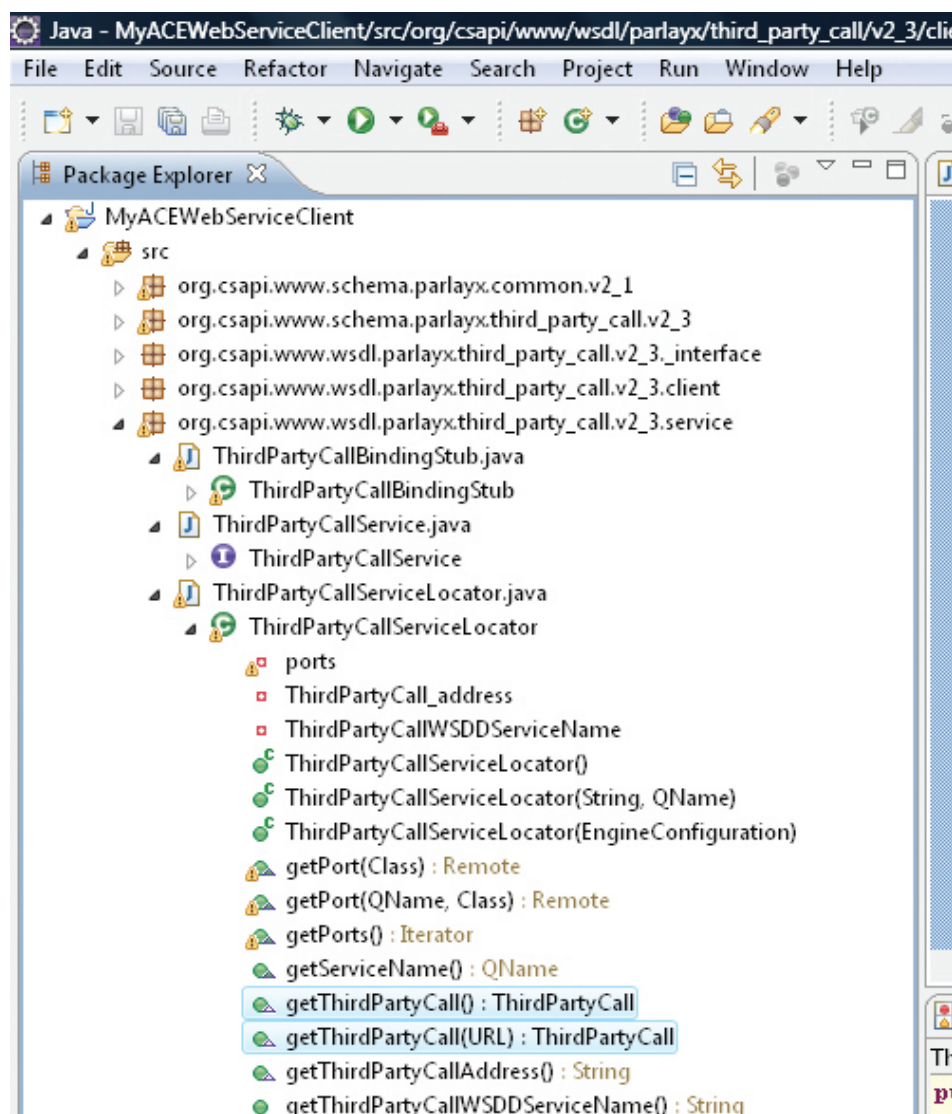


Figure 5.6: Third party call service locator classes

Clients can use the `ThirdPartyCallServiceLocator` class to create a new `ThirdPartyCallServiceLocator` object. The client then calls one of the `getThirdPartyCall` methods on the `ThirdPartyCallServiceLocator` to create an instance of the third party call service interface which it can then use to send requests. The `getThirdPartyCall(URL)` method causes requests to be sent to the location defined by the URL, which is the location of the Web service hosted on the Avaya ACE web server in your development configuration. In practice, it makes sense to supply the URL from a properties file to avoid hard coding it.



If you use the no argument version of `getThirdPartyCall`, the client will attempt to send requests to the default Web service location defined in the WSDL. **This will not work: you must define the location of the Web service on your Avaya ACE server.**

Before we go on to look at the last of the four packages that were generated by Apache Axis, let's take a short detour to investigate the actual code generated for the proxy interface, `ThirdPartyCall`, as shown in **Figure 5.7**. One thing to notice is that all of the methods throw two types Parlay X specific exception: `PolicyException` and `ServiceException`. In fact, all Avaya ACE Web services throw these exceptions, even those which are not Parlay X compliant.


```

/**
 * ThirdPartyCall.java
 *
 * This file was auto-generated from WSDL
 * by the Apache Axis 1.4 Apr 22, 2006 (06:55:48 PDT) WSDL2Java emitter.
 */

package org.csapi.www.wsdl.parlayx.third_party_call.v2_3._interface;

public interface ThirdPartyCall extends java.rmi.Remote {

    public java.lang.String makeCall(org.apache.axis.types.URI callingParty,
        org.apache.axis.types.URI calledParty,
        org.csapi.www.schema.parlayx.common.v2_1.ChargingInformation charging,
        org.csapi.www.schema.parlayx.common.v2_1.SimpleReference reference)
        throws java.rmi.RemoteException,
        org.csapi.www.schema.parlayx.common.v2_1.PolicyException,
        org.csapi.www.schema.parlayx.common.v2_1.ServiceException;

    public org.csapi.www.schema.parlayx.third_party_call.v2_3.CallInformation
        getCallInformation(java.lang.String callIdentifier)
        throws java.rmi.RemoteException,
        org.csapi.www.schema.parlayx.common.v2_1.PolicyException,
        org.csapi.www.schema.parlayx.common.v2_1.ServiceException;

    public void endCall(java.lang.String callIdentifier)
        throws java.rmi.RemoteException,
        org.csapi.www.schema.parlayx.common.v2_1.PolicyException,
        org.csapi.www.schema.parlayx.common.v2_1.ServiceException;

    public void cancelCallRequest(java.lang.String callIdentifier)
        throws java.rmi.RemoteException,
        org.csapi.www.schema.parlayx.common.v2_1.PolicyException,
        org.csapi.www.schema.parlayx.common.v2_1.ServiceException;
}

```

Figure 5.7: Third party call proxy interface code

Service exceptions are thrown when a Web service is unable to process a request, and where resubmitting the request would result in the same failure. A Service Exception would be returned by a Web service when, for example, a request is submitted with invalid input data. Service Exception message identifiers are prefixed “SVC” followed by a numeric value that indicates the cause of the exception.

Policy Exceptions are thrown when requests fail to meet the Web service’s policy criteria. That is to say, the input to the request may be valid but processing the request would break rules or policies on the server-side. Policy Exception message identifiers are prefixed “POL”.

The Service and Policy exception classes are generated in the `org.csapi.www.schema.parlayx.common.v2_1` package, as shown in **Figure 5.8**.

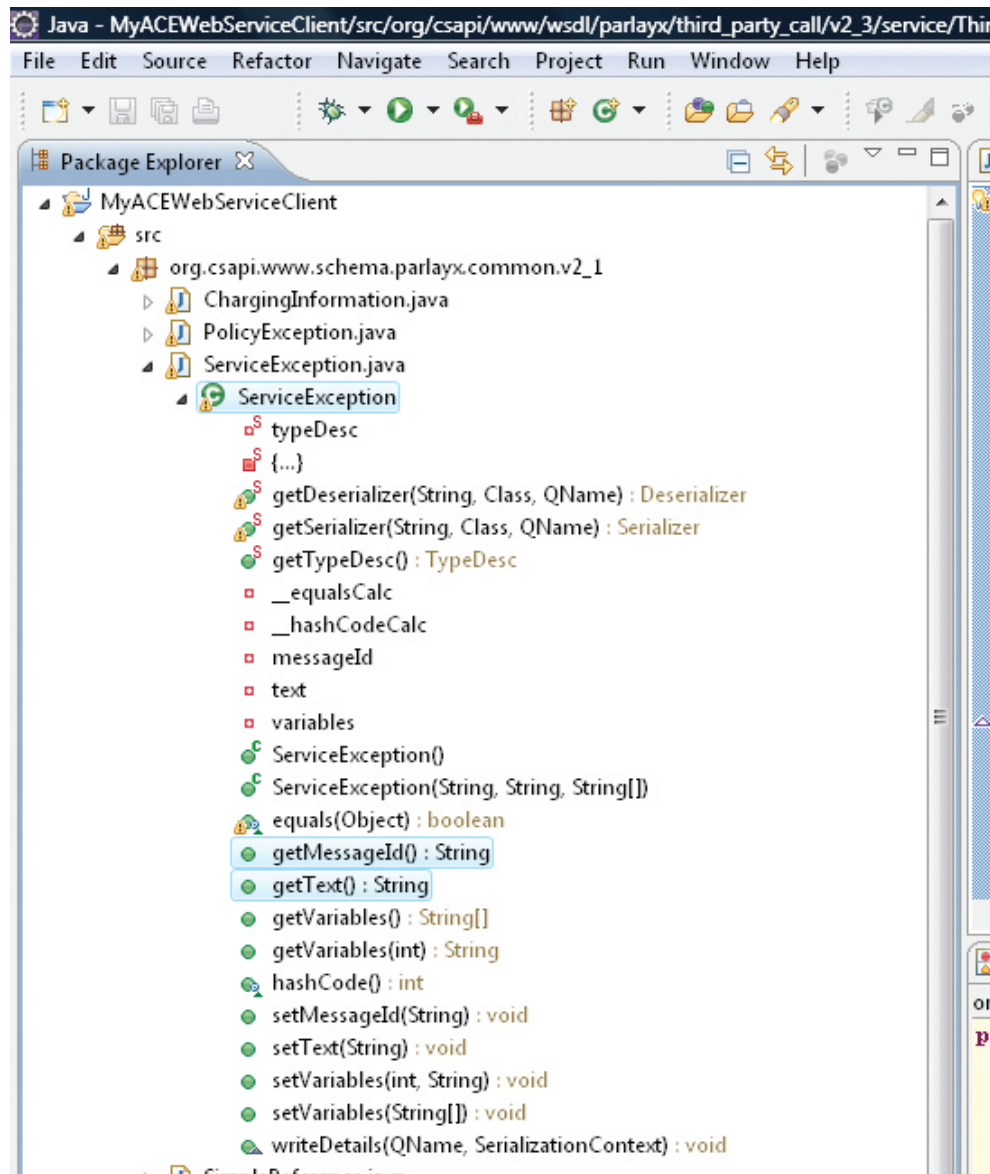


Figure 5.8: Shared fault exception classes

In practice, most of the Avaya ACE Web service methods only throw Service Exceptions. These exceptions are returned to the client in the form of SOAP fault messages, which are then converted into `ServiceException` objects. `ServiceException` provides two key methods to help clients determine the root cause of a failed request and handle it as elegantly as possible:

- `getMessageId()` returns the message identifier.
- `getText()` returns the message text describing the exception.

We'll look at the use of these methods to help handle exceptions in the next section.

Writing client application code

This section takes you step-by-step through the process of writing a client-side Java method that can be triggered, for example, by an `onclick` action on a web page or automatically by a business processing event. The method takes two input parameters, namely strings containing the ACE user names of the two parties to be put into a call. In practice, the ACE user names could be entered in a form, scraped from link text on a web page or retrieved from a database.

The client method generates a `makeCall` operation SOAP request which it sends to the Third Party Call Control Web service hosted on an Avaya ACE server. If the request is successful, the method prints out some information about the call, including its call identifier and the call status; if the request fails, the method handles the exceptions returned by the Web service.

To start things off:

1. Create a new Java class in Eclipse, called `MakePhoneCall`,
2. Import the classes that were generated by Apache Axis and that will be used by the client application.
3. Define the class's constructor method, `MakePhoneCall`, and its arguments.

Figure 5.9 shows the required imports and skeleton code for the `MakePhoneCall` class.

```

import java.util.Calendar;
import org.csapi.www.schema.parlayx.common.v2_1.ServiceException;
import org.csapi.www.schema.parlayx.third_party_call.v2_3.*;
import org.csapi.www.wsdll.parlayx.third_party_call.v2_3._interface.ThirdPartyCall;
import org.csapi.www.wsdll.parlayx.third_party_call.v2_3.service.ThirdPartyCallServiceLocator;

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {
            //TODO
        }

        catch (ServiceException se) {
            //TODO
        }
    }
}

```

Figure 5.9: Create a new client Java class

4. Locate and create the Third Party Call Control service (tpcc), as shown in **Figure 5.10**

As you will see shortly, the service object is used by the client to communicate with the Web service and access its capabilities. Creating the service involves first creating a new `ThirdPartyCallControlServiceLocator` and then calling the `getThirdPartyCall` method on it. For simplicity, we'll pass the location of the Web service directly into the `getThirdPartyCall(URL)` method, rather than retrieve it from a configuration file. The format of the Web service URI depends on whether or not the application wishes to communicate securely with Avaya ACE:

`https://<ace server>:9443/RaptorWeb/services/ThirdPartyCall` or

`http://<ace server>:9080/RaptorWeb/services/ThirdPartyCall`

where `<ace server>` is the IP address of the ACE server hosting the Web service.

```

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {
            //Convert the location of the Web service to a java URL object
            java.net.URL wsURL = new
                java.net.URL("http://10.11.12.130:9080/RaptorWeb/services/ThirdPartyCall")

            // Use the ThirdPartyCallServiceLocator to create a new instance of ThirdPartyCall.
            ThirdPartyCall tpc = new ThirdPartyCallServiceLocator().getThirdPartyCall(wsURL);

            // TODO
        }

        catch (ServiceException se) {
            //TODO
        }
    }
}

```

Figure 5.10: Locate the Third Party Call Control service

Access to Avaya ACE Web services is controlled using HTTP 1.1 basic authentication. This means that all requests to the Web service must contain the username and password set up for the client application's user profile on the Avaya ACE user profile database. The client application's user profile must belong to a user group that is configured with an appropriate access policy to invoke the Web service – see *Chapter 1* for information about Avaya ACE user profiles.

5. Populate the service request with the username and password of the Avaya ACE client application user, as shown in **Figure 5.11**.

```

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {
            ...

            // Use the ThirdPartyCallServiceLocator to create a new instance of ThirdPartyCall.
            ThirdPartyCall tpc = new ThirdPartyCallServiceLocator().getThirdPartyCall();

            // Set username and password to be passed with the request to the Web service
            String userName = new String("My3PCCApp");
            String passWord = new String("Abc12345!");
            org.apache.axis.client.Stub tpcstub = (org.apache.axis.client.Stub)tpc;
            tpcstub.setUsername(userName);
            tpcstub.setPassword(passWord);

            // TODO
        }

        catch (ServiceException se) {
            //TODO
        }
    }
}

```

Figure 5.11: Populate Third Party Call Control service with security credentials

As shown earlier in **Figure 5.7**, the service's `makeCall` method has four arguments:

- `callingParty`
- `calledParty`
- `charging`
- `reference`

The `charging` and `reference` parameters are not used and so can be set to `null`. The `callingParty` and `calledParty` parameters are Java URI objects, so the ACE user names passed to our `MakePhoneCall` method have to be converted from strings to URIs. In this case, we'll convert the strings to ACE format URIs.

6. Convert the calling and called party user names to URIs and pass them to the service's `makeCall` method, as shown in **Figure 5.12**.

```

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {

            ...
            tpcstub.setPassword(passWord);

            // Create URIs for calling and called party ACE IDs, to be put into a call
            org.apache.axis.types.URI callingParty =
                new org.apache.axis.types.URI("ace", callingPartyAceld);
            org.apache.axis.types.URI calledParty =
                new org.apache.axis.types.URI("ace", calledPartyAceld);

            // Put the parties into a call using the Web service's makeCall operation.
            String callId = tpc.makeCall(callingParty, calledParty, null, null);

            // TODO
        }

        catch (ServiceException se) {
            //TODO
        }
    }
}

```

Figure 5.12: Create party URIs and pass into Web service makeCall method

If the request is processed successfully, the Web service returns the unique identifier that it assigns to the call. The call identifier can be used subsequently to control the call and obtain information about it. We'll pass the call identifier into the service's `getCallInformation` method to get a `CallInformation` object that contains information about the call. We'll then call the `getCallStatus.getValue()` method on this object to retrieve the current status of the call. The status is returned just to demonstrate use of the `getCallInformation` method - the status is probably going to be "CallInitial" because the call has just been created. Other possible statuses are "CallConnected" and "CallTerminated".

7. If the Web service returns a call identifier, use it to retrieve the status of the call and print the call identifier and its status to the Java console, as shown in **Figure 5.13**.

```

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {
            ...

            String callId = tpc.makeCall(callingParty, calledParty, null, null);

            // Check whether the request was successful
            if (callId != null) {
                // If the call was successful, get information about the call and print out the call id and status
                CallInformation callInfo = tpc.getCallInformation(callId);
                CallStatus callStatus = callInfo.getCallStatus();
                System.out.println("Call: " + callId + " status: " + callStatus.getValue());
            }
            else
                System.out.println("Call could not be connected.");
            return;
        }

        catch (ServiceException se) {
            //TODO
        }
    }
}

```

Figure 5.13: Check if call was successful and get call information

Finally, we need to define how we want the client to behave if something goes wrong and the Web service returns an exception. The *Avaya ACE Web Services* guide ^[2] tells us that if a `makeCall` operation is unsuccessful, the Web service returns one of the following fault codes:

- **SVC001:** indicates that a service error occurred. The actual error is indicated by an error code in the message text. The meanings of the error codes, and suggested remedial actions, are described in the *Avaya ACE Error Messages Reference* documentation. ^[7]
- **SVC002:** indicates that there was an error in the input data sent with the request. The message text indicates which parameters contained invalid data.

So what we'll do is catch all Service Exceptions and call the `getMessageId` method on the `ServiceException` object to determine whether it's a service or data error. If it's a service error, we'll print out the error code and advise the reader to consult the *Error Messages Reference* documentation; if it's a data error, we'll just print out the message text.

8. Catch and handle Service Exceptions, as shown in **Figure 5.14**.


```

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {
            ...
        }

        catch (ServiceException se) {
            String exceptionMessage = se.getMessageId();
            String exceptionText = se.getText();
            if (exceptionMessage.equals("SVC001"))
                System.out.println("Service error with error Id: " + exceptionText +
                    ". See Avaya ACE Error Management documentation for cause and remedial action." );
            else
                System.out.println("Service exception: " + exceptionMessage + ": " + exceptionText);
            return;
        }
    }
}

```

Figure 5.14: If unsuccessful, handle Service Exceptions from the Web service

And that's just about all there is to it. **Figure 5.15** shows the complete client code for our `MakePhoneCall` method. All that remains is to build, deploy and test the client in the normal way.

```

import java.util.Calendar;
import org.csapi.www.schema.parlayx.common.v2_1.ServiceException;
import org.csapi.www.schema.parlayx.third_party_call.v2_3.*;
import org.csapi.www.wsdl.parlayx.third_party_call.v2_3_interface.ThirdPartyCall;
import org.csapi.www.wsdl.parlayx.third_party_call.v2_3.service.ThirdPartyCallServiceLocator;

public class MakePhoneCall {
    MakePhoneCall(String callingPartyAceld, String calledPartyAceld) {
        try {
            java.net.URL wsURL = new
                java.net.URL("http://10.11.12.130:9080/RaptorWeb/services/ThirdPartyCall")
            ThirdPartyCall tpc = new ThirdPartyCallServiceLocator().getThirdPartyCall(wsURL);

            String userName = new String("My3PCCApp");
            String passWord = new String("Abc12345!");
            org.apache.axis.client.Stub tpcstub = (org.apache.axis.client.Stub)tpc;
            tpcstub.setUsername(userName);
            tpcstub.setPassword(passWord);

            // Create URIs for calling and called party ACE IDs, to be put into a call
            org.apache.axis.types.URI callingParty =
                new org.apache.axis.types.URI("ace", callingPartyAceld);
            org.apache.axis.types.URI calledParty =
                new org.apache.axis.types.URI("ace", calledPartyAceld);

            // Put the parties into a call using the Web service's makeCall operation.
            String callId = tpc.makeCall(callingParty, calledParty, null, null);

            if (callId != null) {
                // If the call was successful, get information about the call and print out the call id and status
                CallInformation callInfo = tpc.getCallInformation(callId);
                CallStatus callStatus = callInfo.getCallStatus();
                System.out.println("Call: " + callId + " status: " + callStatus.getValue());
            } else
                System.out.println("Call could not be connected.");
            return;
        }
        catch (ServiceException se) {
            String exceptionMessage = se.getMessageId();
            String exceptionText = se.getText();
            if (exceptionMessage.equals("SVC001"))
                System.out.println("Service error with error Id: " + exceptionText +
                    ". See Avaya ACE Error Management documentation for cause and remedial action. ");
            else
                System.out.println("Service exception: " + exceptionMessage + ": " + exceptionText);
            return;
        }
    }
}

```

Figure 5.15: Putting it all together

Running the example client application code

When you run the example client application code above, by calling the `MakePhoneCall` method with arguments “absmith” (calling party’s unique ACE user name) and “cdbrown” (called party’s ACE user name), and provided the call is created successfully, the following is displayed in the Java console:

Call: cd23b9a1-5370-4fa7-93ea-4314251d9f7f status: CallInitial

The call flow is shown in **Figure 5.16**. Note that once control passes to the underlying communications infrastructure, it determines how the call is actually set up; this part of the call flow is not shown.

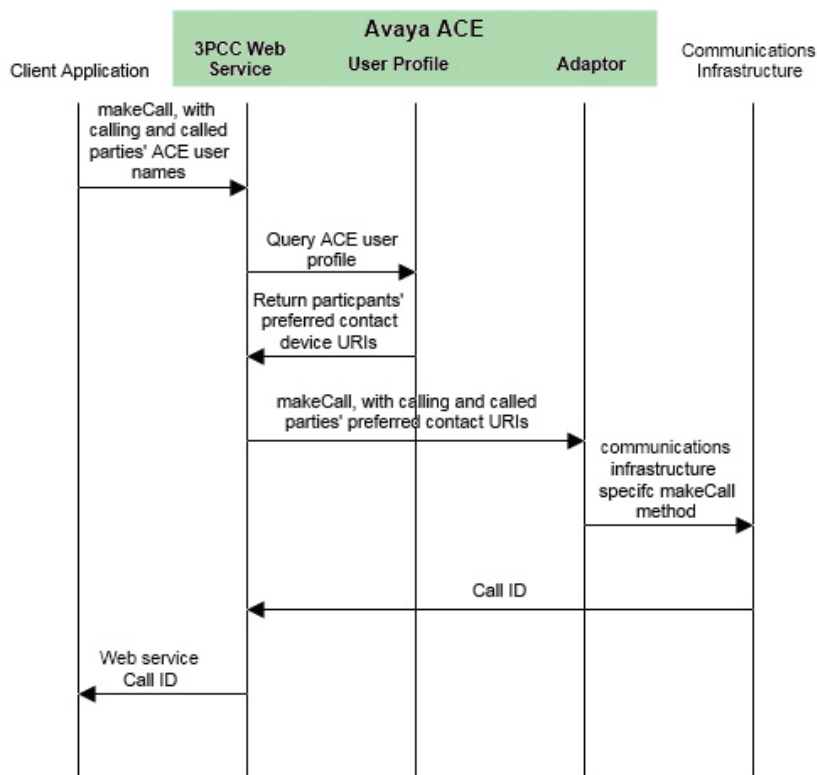


Figure 5.16: Third Party Call Control makeCall call flow

Round-up of Chapter 5

In this chapter, you learned about the options available for accessing an Avaya ACE development configuration, how to obtain Avaya ACE Web service WSDL, how to generate a client proxy from the WSDL and how to develop client applications that make use of the generated proxy code.

The next chapter provides an in depth look at some of the most useful, feature rich Avaya ACE Web services and provides practical advice on how best to leverage their communications capabilities in your business applications.

CHAPTER 6

WEB CLIENT DEVELOPMENT KNOW-HOW

Now that you have learned about the general technical aspects of how to develop Avaya ACE Web service client applications, it's time to look at some real-life examples, and acquire practical know-how that you can apply to your application development.

This chapter uses a couple of scenarios to demonstrate how the Avaya ACE Web services can be used to provide solutions that satisfy realistic business requirements. Stepping through these solutions provides a deeper understanding of the capabilities of some of the most functionally rich Web services, describes how those Web services are used, and gives practical hints and tips for developers. The descriptions of the solutions are in the form of narrative tutorials, rather than low level code samples.

The chapter comprises descriptions of possible solutions for the following scenarios:

Redirect Unanswered Calls: in this scenario, whenever a call is made to a member of an organization's IT Support team during normal office hours and the call is not answered, it is automatically redirected to the team's supervisor. The solution primarily demonstrates the use of the Call Notification Web service, but also uses the Third Party Call Extension Web service.

First Aid Personnel Presence Status: in this scenario, an organization monitors the presence of qualified first aid personnel on its staff and uses the information to maintain a web page on its intranet showing the presence status of the personnel in each of its buildings. The Office Manager can use the web page to check that there is adequate first aid cover in all locations at all times. The web page also includes click-to-dial links which can be used by staff to minimize the time taken to contact an available first aid responder when an accident occurs. The solution primarily demonstrates the use of the Presence Web service, and in particular the Presence Consumer and Presence Notification interfaces.

Redirect unanswered calls solution

In the solution for the Redirect Unanswered Calls scenario, the client application has to:

1. **Start call notification:** subscribe to be notified when incoming calls are not answered at any of the operators' deskphones.
2. **Receive call notifications:** receive the "No Answer" notifications from Avaya ACE.
3. **Redirect calls:** redirect calling parties whose calls are not answered to the team supervisor's deskphone.
4. **Stop call notification:** at the end of the day, unsubscribe to stop being notified of unanswered calls.

Let's look at each of these steps in detail.

Start call notification

The client application uses the Avaya ACE Call Notification Web service to subscribe to and receive notifications of specified call events at specified addresses. We'll use version 3.8 of the Call Notification Web service as it's the one Avaya recommends for new development.

To start call notification, the client application sends a subscription message to the Call Notification Web service, requesting that it be notified of "No Answer" call events at the tel URI addresses of the IT Support team's deskphones. To do this, the client invokes the `StartCallNotification` operation on the Call Notification Manager interface. As defined in the Call Notification Manager WSDL, the `StartCallNotification` operation takes the following parameters:

- **reference:** defines where the requested call event notifications are to be sent.
- **addresses:** the addresses of the devices upon which call events are to be notified.
- **criteria:** the types of call events to be notified.
- **direction:** defines whether call events are to be notified for incoming or outgoing calls.

It's worth looking at each of these parameters in more detail to get a better understanding of how to use the Call Notification Web service and of how call notification works.

Reference parameter

The `reference` parameter defines where the client application wants the call event notifications to be sent and processed. The `reference` has three elements:

- **endpoint:** the URL of the endpoint to which the call event notifications are to be sent.
- **interface:** the name of the port type or client stub on the client that will receive and subsequently process the call event notifications.
- **correlator:** uniquely identifies the subscription request.

The first thing to notice is that the **endpoint** URL is not necessarily the URL of the client that is sending the subscription request; the client can ask the Web service to send the notifications to any endpoint capable of consuming them. This provides a great deal of flexibility and allows for complex solutions where notifications from different subscription requests can be processed in different ways at different endpoints. However, in our Redirect Unanswered Calls solution, the client application will both subscribe to and consume the call notifications.

Call event notifications are in form of SOAP requests sent over HTTP from the Call Notification Web service to an **interface** on the client **endpoint**. In effect, any client applications you create that consume notifications are Web services that process requests from the Call Notification Web service. We'll discuss the implications of this when we get on to how the client receives call event notifications.

The second thing to notice is that the **correlator**, which uniquely identifies each subscription request, is defined by the client. In addition, the correlator supplied in the request must be unique across all client applications that send subscription requests to the Call Notification Web service. One recommended and commonly used format for correlators is "localid@host", where "localid" is a sequential value generated by the client and "host" is the URI of the client. If a new subscription request is sent with an existing correlator, it will fail.

The correlator can be used by the client in a couple of ways:

- to identify the subscription in a subsequent request to stop receiving the call event notifications.
- to support stateful references. As you will see later, the correlator is returned in all call event notification messages from the Web service and so can be used by the client to identify the related subscription for each notification.

Addresses parameter

The **Addresses** parameter contains the list of the directory numbers at which the client wants to monitor call events. The directory numbers are supplied in tel URI address format; the Avaya ACE Call Notification Web service does not support SIP or ACE URIs. In our solution, the Addresses parameter contains the tel URIs of all the IT Support team's deskphones.

Note: if a call has been created using the Third Party Call Control Web service v2, its call identifier can be supplied for this parameter instead of a list of addresses. The client will then be notified of events for that whole call, not just for one of the participant addresses. The **Direction** parameter must not be set when using this option.

Criteria parameter

The **Criteria** parameter contains the list of the call events that the client wants to be notified of. Possible values are “Answer”, “Busy”, “CalledNumber”, “Disconnected” and “NoAnswer, or the parameter can be set to null to receive notifications for all supported call events. To minimize network traffic and preserve bandwidth, it is best to only subscribe to those events the solution is interested in. So, for the Redirect Unanswered Calls solution, the **Criteria** parameter contains just the “NoAnswer” value.

Direction parameter

The **Direction** parameter defines whether the client wants to receive notifications of calls made to the listed **Addresses** (value “Called”) or from the listed **Addresses** (value “Calling”). In our solution, we are only interested in calls made to the IT Support team, so the **Direction** parameter value is set to “Called”.

When the client sends a successful subscription request, the Call Notification Web services returns an empty `StartCallNotificationResponse` message and begins sending “NoAnswer” call event notifications whenever they occur at the monitored addresses.

Receive call notifications

Whenever a call goes unanswered at one of the IT Support team’s deskphones, the Call Notification Web service sends a `NotifyCallEvent` SOAP message to the client application, contain the following information:

- **correlator:** uniquely identifies the subscription the notification relates to. This is the identifier created by the client and included in its original `StartCallNotification` request.
- **eventNotified:** comprises two elements: `CallEvent` is the type of call event being notified, plus an optional Description of the event. In our solution, the `CallEvent` value should always be “NoAnswer”.
- **callingParticipant:** the address from which the call is being made, in tel URI format.
- **callingParticipantName:** the name of the participant making the call. This value may or may not be provided by the Web service, depending on the underlying communications network elements.

- **calledParticipant:** the address to which the call is being made, in tel URI format. The value is usually identical to the one of the **Addresses** in the original subscription request, but in certain circumstances may be returned in a different format – see the *Avaya ACE Web Services guide* ^[1] for more information. In our solution, this is the directory number of the member of the IT Support team.
- **callSessionIdentifier:** identifier that uniquely identifies the call, and which can be used subsequently to control and get information about the call.

As mentioned earlier, the client endpoint must provide an interface that is capable of consuming `NotifyCallEvent` messages from the Web service. This may sound ominous at first, but remember the interface is defined in the Call Notification Web service WSDL. You can use the same development tools that are used to generate the client code needed to create SOAP requests to generate the interface and proxy you need to receive SOAP requests. Some of these tools are described and demonstrated in *Chapter 5*.

To be able to consume the notifications, the client interface has to define a `notifyCallEvent` method with the following signature:

```
notifyCallEvent(String correlator, EventDescription eventNotified,
URI callingParticipant, String callingParticipantName, URI
calledParticipant, String callSessionIdentifier)
```

In effect, the Call Notification Web service notifies the call event by invoking this method on the interface. If the notification is consumed successfully, the client returns an empty `NotifyCallEventResponse` message to the Web service.



Note that if the client fails to consume the call notifications for any reason, ACE automatically terminates the original call notification subscription after a configured number of attempts.

To process the notification and, in our solution, direct the call to the IT Support team supervisor, the client application has to implement the `notifyCallEvent` interface method.

Redirect calls

In our Redirect Unanswered Calls solution, the implementation of the `notifyCallEvent` method uses the Avaya ACE Third Party Call Extensions Web service version v.2.3 to redirect calls to the IT Support team supervisor. The implementation:

1. Checks the value of the `CallEvent` element in the `eventNotified` argument.
2. If the value is “NoAnswer”, locate and create an instance of the Third Party Call Extensions service. *Chapter 5* includes example code which demonstrates how to do this.

3. Populate the service request with the ACE username and password of the client application. Again, see *Chapter 5* for information about how and why this is done.
4. Invoke the `redirectCall` operation on the Third Party Call Extensions Web service, passing in the following parameters:
 - ◇ **callIdentifier:** this is the value provided in the `notifyCallEvent` method's `callSessionIdentifier` argument.
 - ◇ **deviceAddress:** this is the URI of the IT Support team member, provided in the `notifyCallEvent` method's `calledParticipant` argument. Note that this parameter supports SIP, tel and ACE URIs, but in our solution will always be a tel URI.
 - ◇ **destinationAddress:** this is the URI of the supervisor's deskphone.

When a “NoAnswer” event notification is received by the client, the `notifyCallEvent` method is automatically invoked and the call redirected to the supervisor.

Of course, it is always a good idea to anticipate what might go wrong and handle issues as elegantly as possible. For example, if the calling party hangs up immediately after the “NoAnswer” call event notification is sent, the redirect will fail. Some solutions might want to log information about lost calls if they are not answered by the original called party or by the supervisor.

Stop call notification

The solution requires that, at the end of the working day when the supervisor goes home, unanswered calls are no longer redirected. To end the subscription and thereby stop receiving “NoAnswer” call event notifications, the client invokes the `StopCallNotification` operation on the Call Notification Web service's Call Notification Manager interface. The operation takes a single parameter, which is the `correlator` that identifies the subscription.

This completes the description of the Redirect Unanswered Calls solution. The solution illustrates the capabilities of the Call Notification Web service and highlights many of the issues you will need to consider when you use it.

First aid personnel presence solution

Just to remind you, in this scenario an organization monitors the presence of qualified first aid personnel on its staff and uses the information to maintain a web page on its intranet showing the presence status of first aid personnel in each of its building. The Office Manager can use the web page to check that there is adequate first aid cover in all locations at all times. The web page also

includes click-to-dial links which can be used by staff to minimize the time taken to contact an available first aid responder when an accident occurs. Note that the construction of the web page is outside the scope of this tutorial: AJAX, JSP or similar will be required to handle the publication of updated presence information on a web page.

In the solution for the First Aid Personnel Presence scenario, the client application has to:

1. **Subscribe for presence:** subscribe to be authorized to watch the presence status of the first aid personnel and be able to receive their presence status information.
2. **Start presence notification:** request to be notified of all changes to the presence status of the first aid personnel.
3. **Receive presence status information:** receive notifications from the Presence Web service whenever the presence status of any of the first aid personnel is updated.
4. **Display presence information:** display information about the presence status of the first aid personnel on a web page.
5. **Enable click-to-dial links:** enhance the web page with click-to-dial links for available first aid personnel.

As for the previous scenario, let's look at each of these steps in detail.

Subscribe for presence

Before the client application can receive presence status information about the first aid personnel it must first subscribe to be authorized as a watcher. To do this it invokes the `SubscribePresence` operation on the Presence Web service's Presence Consumer interface. The `SubscribePresence` operation takes the following parameters:

- **watcher:** the URI of the watcher that wants to receive presence status information about the **presentities**. In our solution, the watcher is the client application itself.
- **presentities:** an array of addresses of presentities about whom the watcher wants to receive presence status information. The addresses can be SIP, tel, Sametime or ACE format URIs. In our solution, this parameter lists the ACE URIs of all first aid personnel. This means that the watcher can request aggregated presence information from each of the first aid personnel's contact devices that support presence.
- **attributes:** Avaya ACE always registers the watcher for subscription to all available presence attributes, so this parameter is ignored.

- **application:** not used by Avaya ACE; set to null.
- **reference:** not used by Avaya ACE; set to null.

If the subscription request is successful, the Web service registers for presence information with each specified presentity and returns an empty `SubscribePresenceResponse` message and the watcher is permitted to request presence status information about the presentities.



Note that, if subscription to one or more of the specified presentities is unsuccessful, subscription to all other presentities fails. To reduce the possibility of this happening it may be preferable to subscribe to each presentity separately.

Start presence notification

Next, the client application requests to be notified of each first aid responder's current status and thereafter to be notified whenever the presence status of any of the first aid personnel changes. To do this, the client application invokes the `StartPresenceNotification` operation on the Presence Web service's Presence Consumer interface. The `StartPresenceNotification` operation takes the following parameters:

- **watcher:** the URI of the watcher. This is the same value passed into the `SubscribePresence` operation.
- **presentities:** an array of addresses of presentities about whom the watcher wants to be notified of presence status changes. In our solution, this parameter lists the ACE URIs of all first aid personnel.
- **attributes:** Avaya ACE notifies changes to all presence attributes, so this parameter is ignored.
- **reference:** this performs a similar role to the reference used in Call Notification subscriptions and described earlier in this chapter. The reference defines the **endpoint** URL to which presence notifications will be sent (in this case, the URL of the client application), the name of the **interface** on the client that will receive and subsequently process the presence notifications, and a **correlator** which uniquely identifies this presence notification request.
- **frequency:** defines the maximum frequency at which notifications can be sent by the Web service to the watcher, expressed as a number of milliseconds, seconds, minutes, etc. This does not mean that notifications will be sent at this frequency; if the status of the presentities does not change, notifications will not be sent regardless of this value. Be aware, however, that if the frequency were set to say "1 hour" it could be up to an hour before the watcher is notified that the status of a presentity has changed. This wouldn't be acceptable for our solution, so the

parameter is set to a value of “30 seconds”. Selecting a suitable value for this parameter is a compromise between reducing network traffic whilst ensuring presence information is as up-to-date as is required for the solution.

- **duration:** defines the period for which presence notifications are required, usually expressed as number of hours, days, weeks, months or years. In our solution, the notifications are required for an indefinite period, so a value is not set.
- **count:** defines the maximum number of notifications after which the presence notification will automatically stop. Our solution requires that no limit be set on the number notifications, so the value is either not set or set to “0”.
- **checkImmediate:** defines whether the client wants the Web service to provide presence status information about the presentities as soon as presence notification is started. In this case, the solution requires that presence information be initialized for all of the first aid personnel, so the value is set to `true`.
- **noFail:** not used.

If the request is successful for all presentities, the Web service returns an empty `StartPresenceNotificationResponse` message and the watcher will start to receive notifications whenever the presence status of any of the first aid personnel changes. If the request cannot be established for any of the presentities, the response contains the URIs of each device in each user’s profile for which Avaya ACE could not set up notification, along with the ACE URIs of the presentities.

Receive presence status information

When the presence notification is first established, and subsequently whenever the presence status of any of the first aid personnel changes, the Call Notification Web service sends a `StatusChanged` SOAP message to the client application, containing the following information:

- **correlator:** uniquely identifies the notification request that the status changed message relates to. This is the identifier created by the client and included in its original `StartPresenceNotification` request.
- **presentity:** the ACE URI of the first aid responder whose presence status has changed.
- **changedAttributes:** includes multiple elements that provide information about the current status of the presentity.

The full set of `changedAttributes` elements are described in the *Avaya ACE Web Services* guide ^[1], so will not be listed in full here. However, the **TypeAndValue** element includes the **Communications** sub-element which in turn includes the following values that are particularly relevant for our solution:

- **contact:** the URI of the contact device whose presence status has changed.
- **priority:** the priority assigned to the contact device on the first aid responder's ACE user profile: a value between 0 and 1, where 1 is highest priority.
- **type:** the contact device type taken from the first aid responder's ACE user profile: "Phone", "Chat", "Sms", "Video", "Web", "Email", "Mms", "Conference" or "MeansOther".
- **status:** the current status of the contact device: "On", "Off" or "Busy".

In addition, the **TypeAndValue** element includes the **Activity** sub-element which describes the first aid responder's current activity at the contact device: "ActivityNone" (i.e. not known), "Available", "OnThePhone", "DoNotDisturb". However, in our solution we'll ignore the **Activity** value and allow the first aid responder to be contacted as long as their status is "On".

The client must provide an interface that is capable of consuming `StatusChanged` requests from the Presence Web service. The interface is defined in the WSDL for the Presence Notification service which, as for call notification processing described earlier, can be used to generate the interface and proxy you need to receive the SOAP requests. To be able to consume presence notifications, the client interface has to define a `statusChanged` method with the following signature:

```
statusChanged(String correlator, URI presentity, String
callingParticipantName, URI calledParticipant, PresenceAttribute
changedAttributes)
```

In effect, the Presence Web service notifies the change in presence status by invoking this method on the interface. If the notification is consumed successfully, the client returns an empty `StatusChangedResponse` message to the Web service. The client processes the presence notification message in its implementation of the `statusChanged` interface method.

Display presence information

In our solution, the implementation of the `statusChanged` method gets the ACE URI of the presentity, the URI of the contact device and its current presence status from the method arguments. Only telephone numbers are required, so the client ignores notifications unless the contact device type is "Phone".

The First Aid page on the organization's intranet displays the names of the first aid personnel, but this is not provided in presence notifications. To get the name of the first aid responder that the notification relates to, the client invokes the `queryUserNamesByContactId` operation on the User Profile Web service. The method takes a device URI and type (both provided in the notification) and returns the user name of the first aid responder.

If the presence notification indicates that a first aid responder's presence status has been updated to "On", their entry on the web page is updated to show that they are available and enabled as a click-to-dial link (see below). If the presence status has changed from "On" to "Off" or "Busy", the entry is updated to show the first aid responder is not available and the click-to-dial link is disabled.

Enable click-to-dial links

Chapter 5 included an example that showed how to use the Third Party Call Control Web service to implement a `MakePhoneCall` method that provides click-to-dial functionality, so we won't cover it again here. In our solution, when a first aid responder becomes available, their entry on the First Aid intranet page is updated and the `MakePhoneCall` method is called when a user clicks on their name. If you remember, the method takes two parameters, the ACE URI of the user and the ACE URI of the first aid responder, which the client can derive from the notification.

This completes the description of the First Aid Personnel Presence solution, which illustrates the capabilities of the Presence Web service and highlights some of the technical aspects that you will need to consider when you use it.

Round-up of Chapter 6

This completes our look at the core Avaya Web services and their use in communications enabled business systems. In the next chapter we'll take a quick tour of the available packaged Avaya ACE applications, in particular the Event Response Manager application which exposes a RESTful Web service API that developers can use to communications enable their client applications.

CHAPTER 7

AVAYA ACE PACKAGED APPLICATIONS

You may remember from back in *Chapter 1* that the Avaya ACE architecture includes an optional component known as the ACE Integration Engine™ (AIE). AIE is a software engine that hosts selected packaged Avaya ACE applications and provides a single point of access for the administration of those applications. In addition, Avaya ACE offers a number of other packaged applications and add-ins that provide desktop and browser-based CEBS (Communications Enabled Business Systems) and UC (Unified Communications) capabilities. Avaya ACE applications and add-ins orchestrate the SOAP-based Web services hosted on the ACE Core server.

This chapter incorporates brief descriptions of the functionality provided by each of the Avaya ACE applications and add-ins, and looks in more detail at the Event Response Manager application and its programming interface. The Event Response Manager is specially designed to support CEBS by exposing a RESTful Web service API that developers can use to communications enable their client applications.

In addition to the applications covered in this chapter, Avaya ACE provides UC capabilities via integration with IBM Lotus Sametime and Microsoft Office Communications Server (OCS). See *Chapter 1* for details.

Avaya ACE applications and UC add-ins

ACE Integration Engine applications

Mobility application

Mobility supports enhanced communications capabilities on BlackBerry® smartphones:

- **Mobile Cost Optimizer** allows organizations to reduce their corporate mobile call costs by integrating BlackBerry smartphones with their communications environment. The application instructs the company's PBX to use the enterprise voice network, rather of the mobile network, for outbound calls.
- **Conference Call Dialing** allows users to join or chair conference calls from appointment entries in a calendar, using their BlackBerry.
- **Mobile Hot Desking** allows users to manage the communications devices associated with their ACE user profile, from their Blackberry.

Hot Desking Solution

Hot Desking allows users to manage the communications devices associated with their ACE user profile, from a web browser. The application's browser interface can be used to perform tasks such as setting call forwarding rules, registering temporary devices, and making voice and video calls.

Event Response Manager

Event Response Manager supports automatic and manual team conferencing triggered by business events. Event Response Manager is covered in detail later in this chapter.

The Mobility application and Hot Desking Solution are packaged with, and installed on, the AIE. The Event Response Manager is available separately for installation on the AIE. Microsoft OCS integration also requires AIE.

Other Avaya ACE applications

Personal Assistant

Personal Assistant provides a basic user interface that allows Avaya ACE users to manage preferences and settings on their own Avaya ACE user profile. The application can be accessed through a web browser or, with the addition of a plug-in, Microsoft Office Communicator.

Sales Portal

Sales Portal allows organizations to enable click-to-call functionality on their external, customer-facing web site, and allows customers to see the presence status of employees.

Corporate Directory

Corporate Directory allows organizations to communications enable their corporate directories with click-to-call and presence capabilities.

Message Drop and Blast

Message Drop and Blast is a service that integrates with a Customer Relationship Management (CRM) system to automate voice recording and broadcasting of audio messages to specified recipients. See *Chapter 3* for more information about Message Drop and Blast.

Personal Assistant is included with, and installed on, the Avaya ACE Core server. The Sales Portal, Corporate Directory and Message Drop and Blast applications are available as separate software components.

UC Add-ins

The Avaya ACE UC add-ins are deployed on the ACE user's desktop machine and integrate with Microsoft Outlook 2007 and Microsoft Internet Explorer (versions 6, 7 and 8) to provide basic unified communications capabilities:

- **Outlook add-in** allows users to select mail, contact, calendar, task and journal items in Outlook, and click to initiate a call with the originator of the item.
- **Browser add-in** embeds click-to-call, click-to-IM, import contacts and email functionality into intranet web pages.

Event Response Manager application

The Event Response Manager is a package Avaya ACE application that is deployed on the ACE Integration Engine. The application is specifically designed to support communications enabled business systems by helping organizations respond to and resolve business critical events as quickly and as effectively as possible. It works like this:

1. A client-side application monitors the business system for the occurrence of critical events, such as a share price falling below a particular threshold, the number of available trained staff falling below a specified minimum, or a vital piece of machinery going offline.
2. When one of these events occurs, the client-side application sends an appropriate event trigger to the Event Response Manager.
3. On receipt of the trigger, the Event Response Manager calls each member of the response team associated with the event and plays a prerecorded voice message alerting them that the event has occurred.
4. The Event Response Manager puts the event response team members into a voice conference, so they can discuss and resolve the issue.

The Event Response Manager synchronizes with Avaya ACE to check the availability of event response team members and to ensure that it uses each member's preferred contact details.

In addition, the Event Response Manager maintains, and makes available, historical information about event occurrences and the participants in the associated conferences.

Event Response Manager Administration

The Event Response Manager provides a user interface that allows administrators to:

- Identify ACE end users who can be included in event response teams and tag them according to their roles, skills, and locations.

- Define the members of each event response team, based on individual users or on their role, skill, and location tags.
- Upload prerecorded messages, or text-to-speech messages, to the Media Server.
- Define the events that are being monitored by the client-side system, and associate them with the appropriate response team and message.

As well as conferences being triggered automatically by external events, administrators can trigger event conferences manually from the Event Response Manager user interface.

Generating Event Triggers

So, if you are writing an application that monitors critical business events, how do you send a trigger to the Event Response Manager when one of those events occurs? The answer is to use the Event Response Manager's RESTful Web service API. In practice, this involves generating and sending an empty HTTP POST request to the following URI:

`http://<AIE_server_address>:<port>/erm/events/<event_id>/trigger`

On receipt of the request, the Event Response Manager determines the event response team associated with the **<event_id>** supplied in the URI. The Event Response Manager then makes a call from each team member to the Media Server, which plays the message associated with the event and prompts the member to press * on their device's keypad to join a conference call.

Round-up of Chapter 7

In this chapter you learned about the available packaged Avaya ACE applications and UC add-ins, and the capabilities and functionality they provide to Avaya ACE users. In particular, you learned about the Event Response Manager application and how the RESTful Web service API it exposes can be used to trigger automatic and manual team conferencing when predefined business events occur. If you want to know more about the packaged Avaya ACE applications see the *Avaya ACE Overview* guide ^[6] or download installation, administration and end user documentation from the Avaya Support web site.

CHAPTER 8

NEXT STEPS

Hopefully, reading this eBook has whetted your appetite and you want to learn more about Avaya ACE or get started straight away developing ACE Web service clients. This chapter suggests next steps to help you down the road towards communications enabling your business systems with Avaya ACE.

Join the DevConnect program

An essential first step for anyone who wants to learn more and start developing Avaya ACE Web services clients is to join the Avaya DevConnect program. Registered level membership is free and gives you access to the developer resources, remote labs and support options you need. See *Appendix B* for more information about the DevConnect program, including the additional benefits of higher level membership. Alternatively, if you are not already a member, go to <http://www.avaya.com/devconnect> and join now.

Learn more

Download developer-focused documentation and other developer resources from the DevConnect web portal. General Avaya ACE documentation, including installation, administration, system management, user and ACE application guides are available from the Avaya Support site: <http://www.avaya.com/support>.

Find out which capabilities are supported in your environment

As you learned in *Chapter 2*, not all of the Avaya ACE Web services are necessarily supported in a particular environment. Which Web services are supported depends partially on the underlying communications network. So, for example, Avaya Aura supports a different set of Avaya ACE Web services than Avaya Communication Server 1000 or another vendor's communications platform. Other factors that determine which Web services and capabilities can be used include the supported signaling protocols (SIP and/or TR/87) and the availability of network elements such as a supported Media Server or a Media Resource Control Protocol (MRCP) server.

It is important that you know which Web services are supported in your environment. Ensure you have a good understanding of the environment you are developing for and consult the *Avaya ACE Web Services* ^[1] guide to determine whether the capabilities you want to use can be supported.

Choose your application development tools

Web service clients can be written in just about any modern programming language. Identify and obtain development tools for the programming language of your choice. For example, use Apache Axis to help you develop Java clients, or use Visual Studio for .NET clients. See *Chapter 2* for information about Web service development tools.

Develop and test your client code

To be able to develop and test your client code, you will need:

- Avaya ACE Web services WSDL, which you can download from the DevConnect portal or access from the Avaya ACE GUI.
- Access to an Avaya ACE configuration, either within your own development lab or to the DevConnect Avaya ACE remote lab. DevConnect members can book access to the Avaya ACE remote lab free of charge. During the access period, members can use the remote lab on demand, at any time. See the DevConnect portal for details about the configuration of the remote lab, information about which Avaya ACE Web services are supported, and for instructions on how to book and configure access.

Get no cost technical support

If you have issues or questions relating to Avaya ACE Web service client application development, you can obtain no cost technical support through the DevConnect program. All DevConnect members are entitled to free Tier 1 and Tier 2 technical support; higher level DevConnect members with an approved Avaya ACE solution are entitled to Tier 3 and Tier 4 support hours as part of their membership benefits.

Compliance test your solution (higher level DevConnect members only)

Have your solution compliance tested through the DevConnect program to validate that it interoperates successfully with Avaya products and solutions. When compliance testing is successfully completed, you will receive a letter of compliance and the use of the Avaya Compliant Tested logo. Once your solution has been compliance tested, and depending on your membership level, you may be entitled to various co-marketing benefits with Avaya.

Round-up of Chapter 8

This final chapter of this eBook laid out some of the possible next steps towards communications enabling your business systems with Avaya ACE Web services. Now it's down to you to take that next step and start developing business solutions that streamline workflows and are able to respond immediately to time-sensitive issues in the enterprise.

APPENDIX A

AVAYA ACE WEB SERVICES AND OPERATIONS

The table below lists the Web services provided in Avaya ACE release 2.3, together with the operations supported by each Web service and each operation's parameters. The table also indicates which of the Web services are compliant with the Parlay X specifications.

For detailed information about the Web services available in this or other releases, see the *Avaya ACE Web Services* guide ^[1] for the release you are using.

Web Service	Parlay X?	Operation: <i>parameters</i>
Third Party Call Control v2	Yes	makeCall: <i>callingParty, calledParty, reference</i> getCallInformation: <i>callIdentifier</i> cancelCall: <i>callIdentifier</i> endCall: <i>callIdentifier</i>
Third Party Call Control v3	Yes	makeCallSession: <i>callParticipants, callingParticipantName, reference</i> addCallParticipant: <i>callSessionIdentifier, callParticipant</i> transferCallParticipant: <i>destinationCallSessionIdentifier, sourceCallSessionIdentifier, callParticipant</i> deleteCallParticipant: <i>callSessionIdentifier, callParticipant</i> getCallParticipantInformation: <i>callSessionIdentifier, callParticipant</i> getCallSessionInformation: <i>callSessionIdentifier</i> endCallSession: <i>callSessionIdentifier</i>
Third Party Call Extensions v2.0	No	answerCall: <i>callIdentifier, deviceAddress</i> consultationCall: <i>callIdentifier, deviceAddress, consultationAddress, consultOptions, reference</i> consultationComplete: <i>heldCallIdentifier, consultationCallIdentifier, deviceAddress, consultOptions</i> holdCall: <i>callIdentifier, deviceAddress</i> redirectCall: <i>callIdentifier, deviceAddress, destinationAddress</i> retrieveCall: <i>callIdentifier, deviceAddress</i>

Web Service	Parlay X?	Operation: <i>parameters</i>
Third Party Call Extensions v2.3	No	answerCall: <i>callIdentifier, deviceAddress</i> consultationCall: <i>callIdentifier, deviceAddress, consultationAddress, consultOptions, reference</i> consultationComplete: <i>heldCallIdentifier, consultationCallIdentifier, deviceAddress, consultOptions</i> generateDTMF: <i>callIdentifier, deviceAddress, DTMFDigits</i> holdCall: <i>callIdentifier, deviceAddress</i> redirectCall: <i>callIdentifier, deviceAddress, destinationAddress</i> retrieveCall: <i>callIdentifier, deviceAddress</i> singleStepTransfer: <i>callIdentifier, deviceAddress, transferTo</i>
Call Notification v3.2	Yes	startCallNotification: <i>endpoint, interfaceName, correlator, addresses, criteria</i> notifyCalledNumber: <i>correlator, callingParticipant, callingParticipantName, calledParticipant, callSessionIdentifier</i> notifyAnswer: <i>correlator, callingParticipant, callingParticipantName, calledParticipant, callSessionIdentifier</i> notifyCallEvent: <i>correlator, eventNotified, callingParticipant, callingParticipantName, calledParticipant, callSessionIdentifier</i> stopCallNotification: <i>correlator</i>
Call Notification v3.8	Yes	startCallNotification: <i>endpoint, interfaceName, correlator, addresses, criteria, addressDirection</i> notifyCallEvent: <i>correlator, eventNotified, callingParticipant, callingParticipantName, calledParticipant, callSessionIdentifier</i> stopCallNotification: <i>correlator</i>
Call Forwarding	No	setCallForwarding: <i>callee Address, instructions, condition, forwardingAddress</i> getCallForwarding: <i>targetAddress</i> cancelCallForwarding: <i>targetAddress</i>
Call History	No	registerCallHistory: <i>userId, direction, addresses</i> getCallHistory: <i>address, direction</i> getCallHistoryByProfile: <i>userId, direction</i> deregisterCallHistory: <i>userId, direction, addresses</i> deleteCallHistoryRecord: <i>recordId, username</i>

Web Service	Parlay X?	Operation: <i>parameters</i>
Terminal Location	Yes	getLocation: <i>requester, address, requestedAccuracy, acceptableAccuracy, maximumAge, responseTime, tolerance</i> getTerminalDistance: <i>requester, address, latitude, longitude</i> getLocationForGroup: <i>requester, addresses, requestedAccuracy, acceptableAccuracy, maximumAge, responseTime, tolerance</i> setLocation: <i>address, locationInfo, locationTechnology</i>
Audio Call	Yes	playAudioMessage: <i>callSessionIdentifier, audioURL</i> playTextMessage: <i>callSessionIdentifier, text, language</i> getMessageStatus: <i>correlator</i> endMessage: <i>correlator</i>
Presence	Yes	subscribePresence: <i>watcher, presentities, attributes, application, reference</i> getUserPresence: <i>watcher, presentity, attributes</i> startPresenceNotification: <i>watcher, presentities, attributes, reference, frequency, duration, count, checkImmediate, noFail</i> statusChanged: <i>correlator, presentity, changedAttributes</i> endPresenceNotification: <i>correlator</i> publish: <i>presentity, presence</i> getMyWatchers: <i>presentity, watchedSubscriptionStatus</i>
System Monitoring	No	getSystemInfo: <i>none</i>
User Profile	No	createUserProfile: <i>userName, password, surname, givenName, commonName, nickName, postalAddress, title, description, groupName, buddyName, userGroup, type, contactIdentifier, contactName, priority, pin</i> createUserGroupProfile: <i>userGroupName, type, parentUserGroupName</i> addUserToUserGroup: <i>userName, userGroupName</i> addContactInfoToUserProfile: <i>userName, type, contactIdentifier, priority, contactName, pin</i> addBuddyToBuddyGroup: <i>userName, buddyGroupName, buddyName, buddyType</i> addBuddyGroupToUserProfile: <i>userName, buddyGroupName</i> queryAllUserGroupNames: <i>none</i> queryUserNamesByContactID: <i>contactIdentifier, type</i> queryGlobalSecuritySettings: <i>none</i> queryUserGroupPolicy: <i>userGroupName</i> queryUserGroupProfile: <i>userGroup</i>

Web Service	Parlay X?	Operation: <i>parameters</i>
User Profile (continued)	No	queryUserNamesByUserGroup: none queryUserProfile: <i>userName</i> removeBuddyFromBuddyGroup: <i>userName, buddyGroupName, buddyName</i> removeBuddyGroupFromUserProfile: <i>userName, buddyGroupName</i> removeContactInfoFromUserProfile: <i>userName, contactIdentifier</i> removeUserFromUserGroup: <i>userName, userGroupName</i> removeUserGroupProfile: <i>userGroupName</i> removeUserProfile: <i>userName</i> updateContactInfo: <i>userName, contactIdentifier, priority, contactName, pin</i> updateGlobalSecuritySettings: <i>passwordExpiryPeriod, minPasswordLength, minDifferentCharacters, mixedCase, minAlphaCharacters, minSpecialCharacters, permittedSpecialCharacters, minNumericCharacters</i> updatePersonalData: <i>username, password, surname, givenName, commonName, nickName, postalAddress, title, description</i> changePassword: <i>oldPassword, newPassword</i> updateUserGroupPolicy: <i>serviceName, serviceAccessMode, name, accessMode</i>
Subscriber Management	No	getSubscriber: <i>subscriberName, requestScope, requestOptions</i> getSubscriberByContactID: <i>contactID, requestScope, requestOptions</i>

Web Service	Parlay X?	Operation: <i>parameters</i>
Message Drop and Blast	No	setAuthentication: none clickToDial: <i>clientId, historyId, callingParty, calledParty, notificationEndpoint, correlator</i> leaveCall: <i>clientId, called, turretRequestId</i> getCallIds: <i>clientId</i> getRequestIds: <i>clientId, turretrequestId</i> getCallStatus: <i>clientId, callId</i> getRequestStatus: <i>clientId, turretRequestId</i> endCall: <i>clientId, callId</i> recordAnnouncement: <i>clientId, historyId, callingParty, anncURL, language, notificationEndpoint, correlator</i> reviewAnnouncement: <i>clientId, historyId, callingParty, anncURL, language, notificationEndpoint, correlator</i> messageDrop: <i>clientId, anncURL, duration, callId, language</i> messageDropAndLeave: <i>clientId, anncURL, duration, callId, turretRequestId, wait, language</i> cancelMessageDrop: <i>clientId, callId</i> messageBlast: <i>clientId, historyId, anncURL, duration, blastParties, time, wait, preamble, language</i> cancelMessageBlast: <i>clientId, blastId</i> getMessageBlastStatus: <i>clientId, blastId</i> getUserBlasts: <i>clientId</i> deleteHistories: <i>clientId, historyIds</i> getClientHistories: <i>clientId</i> getHistory: <i>clientId, historyId</i> createEvent: <i>clientId, historyId, event</i> deleteEvent: <i>clientId, historyId, eventIds</i> getClients: <i>clientIds</i> deleteClient: <i>clientId</i> callNotification: <i>callStatus, correlator</i> turretRequestNotification: <i>TurretRequestStatus, correlator</i>

APPENDIX B

AVAYA DEVCONNECT DEVELOPER AND PARTNER PROGRAM

Ready for more?

Join Avaya DevConnect today (www.avaya.com/devconnect) to download free resources for Avaya ACE application development, including programming documentation and FAQs, and to access technical support and remote lab options. Keep reading for additional information on the DevConnect program.

About the DevConnect program

As an application developer, you've already made an investment in the future of intelligent communications solutions for your business. We'd like to help you get more out of that investment, by providing you with the tools and resources to take your business to the next level.

Whether you are simply exploring how Avaya technology can create opportunities for your company and customers, or seeking additional technical know-how and information to aid in your current development efforts, a free DevConnect membership offers:

- Downloadable software development kits (SDKs) and client-side libraries aid in application development and integration.
- Step-by-step tutorials, in-depth training courses and on-demand webinars build skills within your technical community.
- Technical Frequently Asked Questions (FAQs) and community-supported forums assist in knowledge sharing.
- Sample applications jump start your application development efforts by demonstrating how to use key APIs.
- DevConnect eBooks provide an ideal introduction to key communications application development topics.
- Developer conferences, podcasts, newsletters and other developer communications provide ongoing technical awareness.
- Remote labs aid in prototyping and proof-of-concept implementations with minimal up-front investments.
- Evaluation kits and developer editions for emerging Avaya products, allow you to explore new solutions that can give you a competitive advantage or help you deliver superior customer service.

- Interoperability Notes and DevConnect Member Application Notes created by our Solution and Interoperability Test Lab assist in deploying solutions.
- Opportunities to participate in technical beta programs give you a leg up on next generation application development activities.
- And much more...

A sample listing of available DevConnect resources

SDK and API downloads and documentation

Information on APIs and downloads of SDKs for over 30 Avaya products, including:

- Avaya ACE
- Avaya Aura core communications services, including Session Manager, Communication Manager (deployed as a feature server or as an evolution server) and Application Enablement Services (AE Services)
- Avaya Aura applications, including Avaya Aura Conferencing and Avaya Aura Contact Center
- Avaya Communication Server 1000
- Avaya IP Office
- Avaya Voice Portal
- Avaya one-X® Deskphones (Push API and PushSDK)
- Avaya one-X® Web Application SDK

Remote labs

DevConnect members can request access to remote lab installations of Avaya products. Some of the remote labs can be accessed on demand; others are available for fixed, scheduled sessions.

- Avaya ACE remote lab
- Self Service remote lab, featuring Avaya Voice Portal
- Avaya Aura Session Manager remote lab
- Avaya Aura AE Services and Communication Manager remote lab
- Avaya Aura Contact Center remote lab
- Avaya Communication Server 1000 remote lab

Training courses, tutorials and technical webinars

- Over 25 hours of in-depth training courses on Avaya product interfaces, including Avaya Aura AE Services, Dialog Designer and the Avaya one-X Deskphone APIs
- Step-by-step tutorials, including:
 - ◊ Setup, Application Initialization, and Event Monitoring using the AE Services DMCC Java SDK
 - ◊ Designing a Microsoft SQL database connector in Dialog Designer
 - ◊ Avaya one-X Deskphone Push and Browser Applications Setup
- Over 15 on-demand Technical Webinars, including *An Introduction to the Voice Portal and Dialog Designer*

Tools, evaluation kits and developer editions

- Avaya Dialog Designer
- Avaya AE Services DMCC Dashboard
- Avaya one-X® Deskphone XML Designer
- Midsize Business Template (MBT) and MBT Lite, comprising instances of Avaya Aura core communications components on a single server, for use in a development lab

Sample applications

More than 25 sample applications (in addition to those provided with individual SDKs) for Avaya one-X Deskphone applications, AE Services, Dialog Designer and Proactive Contact.

DevConnect eBooks

In addition to this Avaya ACE eBook, the following DevConnect eBooks are available:

- Mastering Device, Media and Call Control Using Avaya DMCC Dashboard
- Developing Exceptional Multi-Modal Customer Experiences
- Mastering Avaya one-X® Deskphone Application Development

Register today

Register today for your free DevConnect membership at www.avaya.com/devconnect.

APPENDIX C

COMPARISON OF PROGRAMMING INTERFACES

The table below provides a comparison of the capabilities and characteristics of the Avaya ACE Web services, the Avaya ACE Foundation Toolkit API and Avaya Aura Application Enablement Services.

	Avaya ACE Web services	Avaya ACE Foundation Toolkit	Avaya Aura Application Enablement Services
Programming interfaces	SOAP-based Web services, most of which comply to Parlay X standard specifications	Foundation Toolkit API Sequenced Template API (higher level abstraction of the Foundation Toolkit API)	JTAPI TSAPI DMCC (Java, .Net and XML) Telephony Web service
Supported programming languages	Language independent	Java	Java (JTAPI and DMCC Java) C++ (TSAPI) .Net (DMCC .Net) Others (DMCC XML and Telephony Web service)
Target developers	IT and Web developers with little or no specialist communications knowledge	Java developers with basic SIP knowledge	Specialist IP communications application developers who require advanced capabilities and fine control

	Avaya ACE Web services	Avaya ACE Foundation Toolkit	Avaya Aura Application Enablement Services
Target applications	CEBS applications and UC clients (e.g. Click-to-call via corporate portal or Microsoft Outlook), basic Contact Center applications, etc.	Applications that make, receive and control SIP calls, and integrate media into those calls. Note that Session Manager supports the routing of the originating and destination sides of SIP calls through a sequence of applications. These Sequenced Applications may be created using the Foundation Toolkit API, or be provided by Avaya Aura Communication Manager or Avaya CS 1000.	Classic Contact Center applications, including CTI screen pops, call recording, and state based call routing. Also UC clients, custom endpoint provisioning clients, etc.
Supported communications platforms	• Avaya Aura Communication Manager deployed as an Evolution server, via Application Enablement Services • Avaya CS 1000 • Avaya CS 2100 • Avaya MCS 5100 • Avaya NES Contact Center, via Meridian Link Services • CS 2000 • Cisco Unified Communications Manager • Tandberg Video Communication Server • IBM Lotus Sametime	• Avaya Aura Session Manager, via Foundation Runtime Services hosted on Avaya ACE	• Avaya Aura Communication Manager deployed as an Evolution server



REFERENCES

All of the documents and other resources listed below are available to registered members from the Avaya DevConnect web portal (<http://www.avaya.com/devconnect>) log in required) or from the Avaya Support web site (<http://www.avaya.com/support>).

1. *Avaya Agile Communication Environment Web Services* is the programmer guide for the Avaya ACE web services. Available for download from DevConnect and Support.
2. *Avaya Agile Communication Environment Administration – IBM Lotus Sametime Integration* guide. Available from Support.
3. *Avaya Agile Communication Environment Administration – Microsoft Office Communications Server Integration* guide. Available from Support.
4. *Avaya Agile Communication Environment Administration* guide provides instructions for the initial configuration, administration and ongoing management of Avaya ACE. Available from Support.
5. *Avaya Agile Communication Environment WSDL Files*: all Avaya ACE web services definition files packaged in a single ZIP file. Available from DevConnect.
6. *Avaya Agile Communication Environment Overview* provides a high level introduction to the design, benefit, services and features of Avaya ACE. Available from DevConnect and Support.
7. *Avaya Agile Communication Environment Error Messages Reference* lists all Avaya ACE service error codes together with descriptions of the errors and suggested remedial actions. Available from DevConnect and Support.
8. *Avaya Aura Foundation Toolkit Overview* provides a high level introduction to the Foundation Runtime Service and Foundation SDK. Available from DevConnect and Support.

About Avaya

Avaya is a global leader in enterprise communications systems. The company provides unified communications, contact centers, and related services directly and through its channel partners to leading businesses and organizations around the world. Enterprises of all sizes depend on Avaya for state-of-the-art communications that improve efficiency, collaboration, customer service and competitiveness. For more information please visit www.avaya.com.



INTELLIGENT COMMUNICATIONS

© 2010 Avaya Inc. All Rights Reserved.

Avaya and the Avaya Logo are trademarks of Avaya Inc. and are registered in the United States and other countries.

All trademarks identified by ®, TM or SM are registered marks, trademarks, and service marks, respectively, of Avaya Inc.

All other trademarks are the property of their respective owners. Avaya may also have trademark rights in other terms used herein.

References to Avaya include the Nortel Enterprise business, which was acquired as of December 18, 2009.

11/10 • MIS5279DEV

avaya.com